

AoI and Energy Tradeoff for Aerial-Ground Collaborative MEC: A Multi-Objective Learning Approach

Fuhong Song¹, Qixun Yang¹, Mingsen Deng¹, *Member, IEEE*, Huanlai Xing², *Member, IEEE*, Yanping Liu³, Xi Yu⁴, Kaiju Li⁴, and Lexi Xu⁵

Abstract—This paper studies the age of information (AoI) and energy tradeoff (AET) problem in an aerial-ground collaborative mobile edge computing system, where a high-altitude platform and an unmanned aerial vehicle (UAV) work together to offer computing services for ground devices (GDs). The AET problem is formulated as a multi-objective optimization problem (MOP) that aims at simultaneously minimizing the total AoI of GDs and total energy consumption of the UAV by optimizing its flight paths and task offloading ratios. Addressing the AET problem poses a significant challenge due to the inherent conflict between the two objectives. The existing methods cannot well address the MOP because they adopt the linear combination to transform an MOP into a single-objective optimization problem using fixed weights (i.e., preferences), ignoring the conflict between objectives. Moreover, user preferences may change over time in dynamic MEC systems. To overcome these challenges, we first build a multi-objective Markov decision process model with a vectorial reward for the AET problem. There are one-to-one relationships between each component of the reward and one of the two objectives. Then, we propose a multi-objective learning algorithm based on proximal policy optimization (PPO), which primarily comprises a training phase and an evolutionary phase. The former adopts multi-objective PPO to iteratively optimize multiple learning individuals, aiming to obtain a nondominated policy set. The latter employs a genetic operator to further improve the quality of each policy in the set. Specifically, the crossover and mutation operators operate at the parameter level of policy networks, avoiding stagnation and premature convergence.

The experiment results validate that the proposed approach obtains a set of excellent nondominated policies and a favorable balance between objectives. Moreover, the proposed approach achieves improvements of at least 39.8%, 2.1%, and 15.3% regarding AoI, energy consumption, and cost compared with several algorithms.

Index Terms—Age of information, mobile edge computing, path planning, proximal policy optimization, reinforcement learning, task offloading, unmanned aerial vehicle.

I. INTRODUCTION

WITH the popularization of Internet of Things (IoT) and the fast development of artificial intelligence, ground devices (GDs) play a significant role in diverse intelligent applications, such as intelligent transportation and environmental monitoring. These applications are usually computing-intensive and delay-sensitive, resulting in a surging demand for computing resources. However, GDs often suffer from low computing capability and restricted battery life, due to small physical sizes and stringent production cost limits. Mobile edge computing (MEC) has been introduced to address the above resource-constrained problem by relocating computing resources to the network edge [1]. Within this paradigm, GDs are empowered to migrate their applications to ground base stations (BSs) situated nearby, thereby enabling them to efficiently support such applications [2]. Nevertheless, the wireless communication coverage of a ground BS is limited and fixed, failing to connect to users located beyond the BS's coverage area. Particularly, BSs may encounter damage due to military attacks or natural disasters, resulting in scarce computing resources and poor service performance [3]. Unmanned aerial vehicles (UAVs) have emerged as an efficient solution for extending communication coverage and enhancing deployment efficiency due to their high mobility and excellent maneuverability [4], [5], [6]. For example, UAVs can provide vital communication support when BS-based infrastructures are either unavailable during disaster rescue operations or sparsely distributed in remote mountainous areas. Equipped with communication and computing resources, UAVs can collect computation tasks from GDs nearby and handle them.

The quality of experience (QoE) is largely influenced by the freshness of collected data in various delay-sensitive applications. For instance, in forest fire monitoring applications, rapid collection and processing of temperature and humidity data are crucial for the accurate prediction of forest fires. In automatic

Manuscript received 17 December 2023; revised 25 March 2024; accepted 22 April 2024. Date of publication 29 April 2024; date of current version 5 November 2024. This work was supported in part by the National Natural Science Foundation of China under Grant 62061007, in part by Guizhou Provincial Science and Technology Project under Grant ZK[2024]693 and under Grant ZK[2023]028, in part by Guizhou Provincial Key Technology R&D Program under Grant QKHZC[2023]YB185, and in part by the Guizhou University of Finance and Economics Introduced Talent Research Start-Up under Grant 2022YJ039. Recommended for acceptance by Haipeng Yao. (*Corresponding author: Mingsen Deng.*)

Fuhong Song, Qixun Yang, Mingsen Deng, Xi Yu, and Kaiju Li are with the School of Information, Guizhou University of Finance and Economics, Guiyang 550025, China (e-mail: fhsong@mail.gufe.edu.cn; qixunyang@mail.gufe.edu.cn; msdeng@mail.gufe.edu.cn; yuxi@mail.gufe.edu.cn; likaiju@mail.gufe.edu.cn).

Huanlai Xing is with the School of Computing and Artificial Intelligence, Southwest Jiaotong University, Chengdu 611756, China (e-mail: hxx@home.swjtu.edu.cn).

Yanping Liu is with the College of Big Data Statistics, Guizhou University of Finance and Economics, Guiyang 550025, China (e-mail: liuyanping6@126.com).

Lexi Xu is with the Research Institute, China United Network Communications Corporation, Beijing 100048, China, and also with the Queen Mary University of London, E1 4NS London, U.K. (e-mail: davidlexi@hotmail.com).

Digital Object Identifier 10.1109/TMC.2024.3394568

drive applications, instantaneous image data handling is imperative to guarantee driving security. In emerging industrial applications, the freshness of sensing data has an important effect on the QoE of users. In these delay-sensitive applications, outdated information may result in erroneous control decisions and catastrophic accidents. To rigorously assess the timeliness of collected data in UAV-assisted MEC systems, a novel criterion known as the age of information (AoI) has been established. This criterion provides a precise quantification of data freshness, enabling the evaluation of system performance [7], [8].

A. Related Work

Considerable researchers have devoted significant attention to investigating diverse aspects of UAV-assisted networks with information freshness. Recently, traditional methods, such as iterative algorithms, linear programming, convex optimization, genetic algorithms, and game theory, have been employed to optimize UAV-assisted networks involving information freshness, yielding satisfactory optimization outcomes. Qin et al. investigated an AoI-based task scheduling problem for aerial-ground collaborative MEC [7]. They proposed an iterative method to minimize the linear combination of each GD's AoI. The researchers in [9] also presented an iterative algorithm to decrease the average AoI of all GDs by planning the UAV's flying paths and bandwidth allocation. Liu et al. investigated the UAV-assisted task collection involving information freshness, and they proposed a two-phase-based iterative method to reduce GDs' maximal AoI and average AoI. In [10], an AoI and energy efficiency tradeoff problem in a UAV-aided network was investigated. The researchers presented a multi-objective mixed integer linear programming approach to obtain nondominated policies of UAV paths [10]. The authors in [11] employed convex optimization and genetic algorithm to plan the UAV's flight paths, aiming at minimizing the UAV's mission completion time, subject to AoI constraints of collected data. To overcome security issues, Yang et al. studied an AoI-aware channel access problem in MEC networks [12]. They proposed a game theory based learning method to achieve channel access schemes. In a follow-up study, they considered the AoI minimization problem in a UAV-assisted traffic monitoring network from the perspective of the Stackelberg game [13]. Han et al. also adopted the game theory to decrease the system's AoI in a UAV-assisted vehicular MEC system [14]. Chen et al. investigated a task offloading and resource allocation problem in an aerial-ground network including a high-altitude platform (HAP) and multiple UAVs [15]. They proposed a distributed online method to lower the energy consumption of all GDs. Jia et al. presented the hierarchical aerial computing system, where HAPs and UAVs worked together to offer computing supports for GDs [16]. They adopted the game theory to maximize the total data amount processed.

An increasing number of researchers adopt reinforcement learning (RL) to optimize UAV-assisted networks involving AoI. To reduce the average AoI, the researchers in [8] presented a deep Q-network (DQN) based approach in a UAV-enabled data collection network. Eldeeb et al. also adopted DQN to lower

the average AoI in IoT networks by planning the UAV's flight paths [17]. Choudhury et al. developed an AoI minimization scheduling method based on DQN in a two-hop UAV-relayed system [18]. The researchers in [19] proposed an actor-critic-based framework to optimize the UAV's paths and data transmission time, aiming at reducing AoI of all GDs. In [20], the researchers considered an AoI-aware UAV-assisted wireless power transfer network. They presented a DQN-based path optimization method to decrease the total AoI of all GDs. Yi et al. studied a fresh data collection problem with wireless charging in UAV-enabled networks [21]. They developed a dueling double DQN (D3QN) approach to decrease the linear combination of the AoI and recharging cost using fixed weights. Chen et al. also introduced a D3QN-based method to optimize the flight paths of UAVs to decrease the energy consumption of GDs, subject to the AoI updating constraints [22]. Samir et al. modeled the UAV flight altitude and communication scheduling problem by Markov decision process (MDP) [23]. They adopted the proximal policy optimization (PPO) method to lower the expected linear combination of AoI. In [24], a UAV path optimization problem was researched in a cluster-based wireless network. The authors presented a transformer-based RL algorithm to minimize the total AoI of collected data. The authors in [25] formulated the UAV-assisted AoI-energy-aware path planning problem as an MDP. They proposed a twin-delayed deep deterministic (TD3) method to lower the linear combination of average AoI and energy consumption. Chen et al. investigated the AoI-oriented task offloading in an aerial-ground MEC network [26]. They proposed an online RL method based on double DQN to minimize AoI. Multi-agent RL methods were adopted to jointly optimize the AoI-based wireless network with multiple UAVs. For example, Wu et al. formulated an AoI-aware sensor data transmission problem by an MDP model [27]. They adopted a multi-agent deep deterministic policy gradient (MADDPG) to reduce the average AoI by planning UAVs' flight paths. Oubbati et al. developed a multi-agent DQN based UAV path planning method to simultaneously minimize the expected AoI and UAVs' energy consumption [28]. The authors in [29] investigated a UAV-aided MEC disaster system to reduce the linear combination of AoI and energy consumption by a multi-agent RL method. Cheng et al. studied the task and energy offloading problem in an HAPs and UAVs assisted MEC network [30]. They presented a soft actor-critic based multi-agent RL approach to maximize the total system utility. Kang et al. studied a hierarchical aerial MEC network consisting of HAPs and UAVs, where UAVs collected tasks from GDs and offloaded them to HAPs [31]. They developed a PPO based multi-agent approach to maximize the amount of completed tasks.

Recently, deep learning has been applied to various fields, such as classification and robot [32], [33], [34]. Jiang et al. presented a multiple graph learning neural networks framework to address multi-view semi-supervised classification problems [33]. The framework aimed at learning an optimal graph structure from a set of graphs. However, the graph learning framework was unsuitable for the problem concerned in this paper. This was because a computation task couldn't usually be represented as a task graph. Singh et al. introduced "Tinku",

TABLE I
COMPREHENSIVE COMPARISON BETWEEN THE RELATED WORKS AND OURS

	Reference	[7]	[8]	[9]	[10]	[11]	[12]	[13]	[14]	[15]	[16]	[17]	[18]	[19]
System model	High-altitude platform									✓	✓			
	Single-objective	✓	✓	✓		✓	✓	✓	✓	✓	✓	✓	✓	✓
	Multi-objective				✓									
Optimization technique	Traditional method	✓		✓	✓	✓	✓	✓	✓	✓	✓		✓	✓
	SORL method		✓										✓	✓
	Multi-objective method													
	Reference	[20]	[21]	[22]	[23]	[24]	[25]	[26]	[27]	[28]	[29]	[30]	[31]	Ours
System model	High-altitude platform											✓	✓	✓
	Single-objective	✓		✓	✓	✓		✓	✓			✓	✓	
	Multi-objective		✓				✓			✓	✓			✓
Optimization technique	Traditional method													
	SORL method	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	✓	
	Multi-objective method													✓

a cost-efficient robot designed for teaching special needs children [34]. Tinku incorporated offline speech processing, computer vision for obstacle avoidance, and anthropomorphic expressions, presenting a promising solution for improving interaction and learning experiences in autism therapy. Although Tinku was a very effective robot for autism spectrum disorder problems, it didn't consider the robot's path planning. The obstacle avoidance method in Tinku could be used for reference in multi-UAV network scenarios.

Table I shows the comprehensive comparison between related works and ours regarding system model and optimization technique. We consider an HAP and UAV collaborative network scenario, where a multi-objective optimization problem (MOP) is investigated. The following section outlines the key distinctions between our work and prior research.

With respect to system model, most aforementioned studies adopt one or more UAVs to provide computing resources in wireless networks. However, a UAV's energy capacity and computing resources are limited, making it challenging to support the computational demands of large-scale tasks from GDs. Although BSs can serve as auxiliary computing platforms, they are often unavailable in extreme scenarios such as natural disasters or military attacks. To address this challenge, some studies concentrate on the integration of HAP into UAV-assisted MEC systems [30]. By establishing effective collaboration between HAP and UAV, the computation tasks collected by UAVs can be offloaded to HAPs for remote processing. On the other hand, a significant portion of existing research focuses on single-objective optimization problems (SOPs) primarily aimed at minimizing AoI, disregarding other crucial objectives. In UAV-assisted networks, for instance, the UAV's energy consumption remains a critical concern. However, the literature only provides a limited exploration of simultaneously minimizing both AoI and energy consumption.

In terms of optimization technique, some studies employ traditional optimization methods such as iterative algorithms [7], [9], convex optimization [11], and game theory [13], [14]. However, these methods struggle to adapt to dynamic network environments, especially when UAVs exhibit rapid mobility and task arrivals are unpredictable. The dynamics and uncertainty often necessitate the repetitive execution of these methods, leading to substantial computational overhead and hampering responsiveness. Unlike traditional methods, RL based methods

can address complex optimization problems using limited prior information by rapidly adjusting their behavior to adapt to dynamic MEC environments. Nevertheless, all existing methods are based on single-objective reinforcement learning (SORL), employing linear aggregation approaches with fixed weights to transform an MOP into an SOP. This overlooks the conflict relationships between objectives, thus failing to obtain balanced optimization results across multiple objectives.

B. Contribution

This paper considers an aerial-ground collaborative MEC system involving information freshness, where an HAP and a UAV cooperate with each other to provide computing services to GDs. The UAV cruises with planned paths to gather computation tasks from GDs. These tasks can be both processed on the local UAV or partly migrated to the HAP for offloading execution when needed. In the proposed system, we study the AoI and energy tradeoff (AET) problem. Unlike previous studies that concentrate on SOP or MOP using the linear combination method, we focus on the simultaneous minimization of two conflicting objectives, i.e., AoI and energy consumption. We propose a novel multi-objective learning (MOL) algorithm to solve the AET problem. The primary contributions of the paper are outlined as follows.

- We consider an aerial-ground collaborative MEC system consisting of an HAP and a UAV. The two work together to offer computing support for GDs. The AET problem is formulated as an MOP problem, and its aim is to simultaneously minimize the total AoI and total energy consumption of the UAV via optimizing the UAV's flight paths and task offloading ratios. Addressing this MOP poses significant challenges due to the inherent conflict between the objectives, making it difficult to strike a balance between them. For example, task collection can decrease AoI while increasing energy consumption.
- We establish a multi-objective Markov decision process model featuring a vectorial reward comprising two components, each of which corresponds to an optimization objective of the AET problem. A multi-objective learning algorithm based on PPO is proposed to address the AET problem, namely MOL-AET. The algorithm mainly consists of the training and evolutionary phases. In the

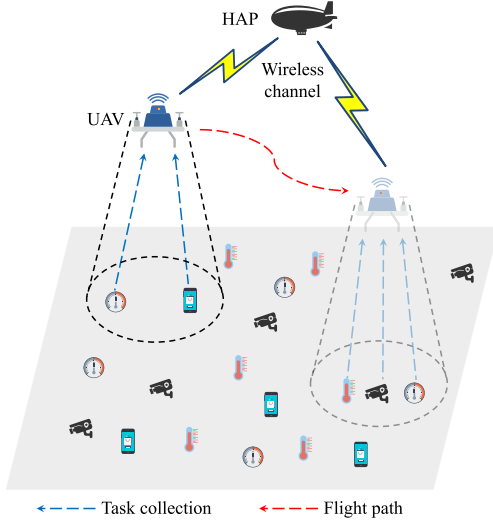


Fig. 1. Aerial-ground collaborative MEC system.

training phase, multiple learning individuals are optimized iteratively by multi-objective PPO. To avoid premature convergence, the evolutionary phase adopts the genetic operator (i.e., crossover and mutation) on each learning individual and its matched individual, aiming at further improving the nondominated policies obtained in the training phase. MOL-AET can output multiple nondominated policies to meet diverse user preferences.

- We perform extensive experiments on a large number of generated test instances, each of which has two different system parameters, i.e., the number of GDs and the UAV's flight height. The results validate that MOL-AET obtains a set of excellent nondominated policies and surpasses two well-known multi-objective evolutionary algorithms and three exclusively developed multi-objective RL algorithms against the inverted generational distance and generational distance. Furthermore, MOL-AET achieves gains of at least 39.8%, 2.1%, and 15.3% regarding average AoI, average energy consumption, and average cost compared with the above five algorithms.

The remainder of this paper is structured as follows. Section II introduces the system model and formally defines the AET problem. To facilitate subsequent analysis, Section III offers a concise overview of MOMDP and MOP. Section IV then delves into the proposed algorithm specifically designed to address the AET problem. Section V presents and analyzes the experimental results obtained, highlighting the effectiveness of the proposed approach. Finally, Section VI concludes the paper and introduces future work.

II. SYSTEM MODEL AND PROBLEM FORMULATION

This paper studies an aerial-ground collaborative MEC system, illustrated in Fig. 1. The system comprises an HAP and a UAV that cooperatively deliver computing services to a set of GDs, represented by the set $\mathcal{J} = \{1, \dots, J\}$. These GDs are randomly scattered in a rectangular area to sense the environmental information, like humidity and temperature. Assume there is

TABLE II
MAIN NOTATIONS USED IN THE SYSTEM MODEL AND PROPOSED ALGORITHM

Notation	Description
Notation used in the system model	
$j, J, \mathcal{J}$	Index, number, and set of GDs
$t, T, \mathcal{T}$	Index, number, and set of time slots
$\mathcal{I}, \mathcal{C}$	Input data size and computation intensity of a task
$A_{\max}$	Maximal AoI that the MEC system can tolerate
A_t	AoI of the MEC system in time slot t
A_{total}	Total AoI of the MEC system during T time slots
A_t^j	AoI of GD j in time slot t
E_t	Energy consumption of the UAV in time slot t
E_{total}	Total energy consumption of the UAV
f_{UAV}	Computing capability of the UAV
g_0	Channel power gain for 1 m reference distance
$\mathcal{N}_t^C$	Number of collected tasks
$\mathcal{N}_t^L$	Number of tasks processed locally on the UAV
$\mathcal{N}_t^O$	Number of tasks offloaded by the UAV
$\mathcal{N}_t^Q$	Number of queued tasks
$\mathcal{N}_t^U$	Number of uncompleted tasks
$\mathcal{N}_{\max}$	Maximal number of tasks the UAV can store
P_{UAV}	Transmission power of the UAV
$R_{\max}$	Maximal horizontal coverage of the UAV
δ	Time duration of a time slot
$\vartheta_{\max}$	Maximal azimuth angle
κ	Effective capacitance coefficient
Notation used in the proposed algorithm	
$a, \mathcal{A}$	Action and action space
$s, \mathcal{S}$	State and state space
$P_{\text{cro}}, P_{\text{mut}}$	Crossover and mutation probabilities
m, n	Number of objectives and learning individuals
M_{str}	Mutation strength
$\mathbf{r}_t$	Vectorial reward
γ	Discount factor
Φ_{evo}	Maximal number of evolution generations
Φ_{tra}	Maximal number of training times

no available ground cellular communication system for GDs, such as a remote area. This paper adopts a rotary-wing UAV for task collection from GDs and subsequent computational support since this kind of UAVs can maintain close proximity to the GDs at a low height. The HAP floats in the stratosphere in a quasi-static manner and its floating height is fixed. The HAP has powerful computing ability and acts as a collaborative computing scheme to handle tasks offloaded by the UAV. All collected computation tasks can be processed either locally on the UAV or partially migrated to the HAP for offloading execution when necessary. In this paper, we employ the time division multiple access protocol for the fresh task collection in the MEC system.

In general, a computation task can be modeled as a two-tuple $\langle \mathcal{I}, \mathcal{C} \rangle$, where $\mathcal{I}$ and $\mathcal{C}$ indicate the input data size and computation intensity of the task, respectively. Note that $\mathcal{C}$ reflects how many CPU cycles are needed to handle one-bit input data. We focus on a discrete-time system, where each time slot has a fixed duration denoted by δ . Assume the whole flight mission (i.e., a task collection period) has T time slots, represented by the set $\mathcal{T} = \{1, \dots, T\}$. Table II summarizes the main notations used in the system model and proposed algorithm.

A. UAV Movement Model

Similar to previous works [3], [35], we employ the Cartesian coordinate system to indicate the spatial positions of the HAP, UAV, and GDs. The UAV cruises with a constant speed and its

flight height U is fixed. We assume that the locations of GDs are static, i.e., their locations are unchanged during the whole flight mission. The coordinates of the HAP and GD j are represented as $\mathcal{M}^H = (X^H, Y^H, H)$ and $\mathcal{M}^j = (X^j, Y^j, 0)$, respectively, where H is the HAP's floating height. The UAV's coordinate is denoted as $\mathcal{M}_t^U = (X_t^U, Y_t^U, U)$ and it may change over time since the UAV flies over GDs to collect their computation tasks. Suppose that the UAV can move with a horizontal direction b_t and a horizontal distance d_t in time slot t . The constraints of the two variables need to be satisfied as follow.

$$0 \leq b_t \leq 2\pi, 0 \leq d_t \leq d_{\max}, \quad (1)$$

where $d_{\max}$ is the maximal flight distance that the UAV can move in each time slot because of the finite energy cost. On the basis of b_t and d_t , the horizontal coordinate of the UAV in time slot $t + 1$ is calculated as

$$\begin{cases} X_{t+1}^U = X_t^U + d_t \cdot \cos(b_t) \\ Y_{t+1}^U = Y_t^U + d_t \cdot \sin(b_t). \end{cases} \quad (2)$$

We assume the UAV operates at a fixed speed $\nu_t = d_t/\delta$, bounded by a predefined maximal flight speed $\nu_{\max}$. In addition, the UAV's movement is confined within a rectangular area with side lengths $X_{\max}$ and $Y_{\max}$.

According to [35], when the rotary-wing UAV flies at speed ν , the corresponding propulsive energy consumption is defined as

$$P(\nu) = \underbrace{P_{\text{bla}} \left(1 + \frac{3\nu^2}{\nu_{\text{tip}}^2} \right)}_{\text{Blade profile}} + \underbrace{P_{\text{ind}} \left(\sqrt{1 + \frac{\nu^4}{4\nu_{\text{rot}}^4}} - \frac{\nu^2}{2\nu_{\text{rot}}^2} \right)}_{\text{Induced power}}^{\frac{1}{2}} + \underbrace{\frac{1}{2}d_0\epsilon\psi\eta\nu^3}_{\text{Parasite power}}. \quad (3)$$

It is observed that $P(\nu)$ comprises the blade profile, induced power, and parasite power. In the blade profile, P_{bla} and ν_{tip} are the blade profile power under hovering status and tip speed of the rotor blade, respectively. In the induced power, P_{ind} and ν_{rot} denote the induced power and mean rotor-induced speed in hovering, respectively. In the parasite power, d_0 , ϵ , ψ , and η represent the fuselage drag ratio, air density, rotor solidity, and rotor disc area, respectively. Based on (3), the energy consumption, E_{fly} , incurred by the UAV during a flight and hovering period of T is obtained by

$$E_{\text{fly}} = \int_0^T P(\nu_t) dt. \quad (4)$$

B. AoI Model

We utilize the AoI indicator to quantify the timeliness of collected tasks. AoI is known as the elapsed time since a GD produces its most recent task [8]. In time slot t , we define the task's AoI of GD j as

$$A_t^j = \delta(t - L_t^j), \quad (5)$$

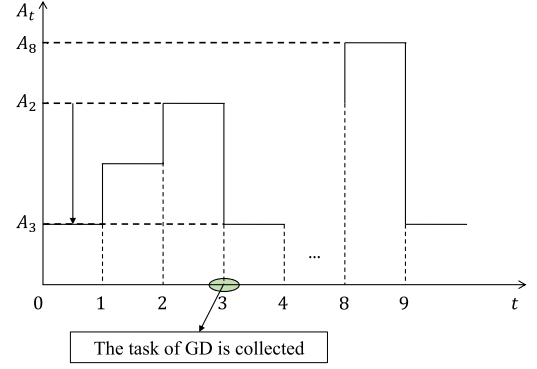


Fig. 2. AoI changing process of a GD.

where L_t^j represents the time when the UAV collects the task from GD j last time.

The range of task collection is limited because the UAV owns limited communication coverage. Thus, the UAV is only able to gather computation tasks from GDs within its coverage. This depends on its maximal azimuth angle $\vartheta_{\max}$ and fixed flight height U . Let $\mathcal{J}_t^C$ denote the set of GDs covered by the UAV in t , given by

$$\mathcal{J}_t^C = \{j | d_t^j \leq R_{\max}, j \in \mathcal{J}\}, \quad (6)$$

where $d_t^j = \sqrt{(X_t^U - X^j)^2 + (Y_t^U - Y^j)^2}$ is the horizontal distance between the UAV and GD j in t . $R_{\max} = U \cdot \tan(\vartheta_{\max})$ is the maximal horizontal coverage of the UAV.

According to whether the task of GD $j \in \mathcal{J}$ is collected, the GD's AoI in $t + 1$ is defined as

$$A_{t+1}^j = \begin{cases} \delta, & \text{if } j \in \mathcal{J}_t^C \\ \min\{A_{\max}, A_t^j + \delta\}, & \text{otherwise} \end{cases} \quad (7)$$

where $A_{\max}$ denotes the maximal AoI that the MEC system can tolerate, and it is the same for all GDs. According to (7), when the UAV collects a task from GD $j \in \mathcal{J}_t^C$, the GD's AoI value decreases to δ ; otherwise, its AoI value increased by δ but the value can not exceed $A_{\max}$. Note that if the AoI A_t^j is greater than the largest AoI $A_{\max}$, the corresponding task of GD j is deemed to be invalid. Therefore, the UAV should follow an appropriate path to collect the computation tasks from GDs in time, lowering the AoI of each GD as much as possible.

Fig. 2 depicts an example of the AoI changing process of a GD. It is seen that the GD generates a computation task in time slot $t = 0$ and its AoI A_0 is initially set to δ . After two time slots, the GD's task is still not collected by the UAV, thus the AoI increases to $A_2 = 2\delta$. In time slot $t = 3$, the UAV collects the GD's computation task, hence the corresponding AoI is reset to $A_3 = \delta$.

C. Computing Model

We consider that the UAV keeps a computing queue in order to store the collected tasks, which are awaiting for further handling. It should be noted that task collection delay and associated energy consumption can be neglected because the UAV can sufficiently approach GDs. The UAV can only store the specified

number of computation tasks due to its limited storage capacity, denoted by $\mathcal{N}_{\max}$. Furthermore, it has to execute the unfinished tasks stored in the computing queue in each time slot. Let $\mathcal{N}_t^U$ represent the number of unfinished tasks in the queue, and its value is an integer limited between 0 and $\mathcal{N}_{\max}$. Denote the task offloading ratio by $o_t \in [0, 1]$ in time slot t . To be specific, $\mathcal{N}_t^O = \lfloor o_t \mathcal{N}_t^U \rfloor$ computation tasks are to be migrated to the HAP for offloading execution, where $\lfloor \cdot \rfloor$ indicates the floor function. The UAV locally executes the remaining $\mathcal{N}_t^L = \mathcal{N}_t^U - \mathcal{N}_t^O$ computation tasks. Within a time slot, the UAV can only process a finite number of computation tasks due to its limited computing capability, denoted by f_{UAV} . Thus, the number of tasks executed by the UAV within a time slot can be obtained by $\min\{\mathcal{N}_t^L, \phi\}$, where $\phi = \lfloor \tau f_{\text{UAV}} / \mathcal{IC} \rfloor$ is the maximal number of tasks that the UAV can process in each time slot.

Based on the above definitions, the number of queued tasks at the end of t is obtained by

$$\mathcal{N}_t^Q = \max\{\mathcal{N}_t^U - \phi - \mathcal{N}_t^O, 0\}. \quad (8)$$

According to (8), the number of tasks removed from the queue is $\phi + \mathcal{N}_t^O$, which includes ϕ processed tasks and $\mathcal{N}_t^O$ offloaded tasks.

At the beginning of $t + 1$, the number of uncompleted tasks, $\mathcal{N}_{t+1}^U$, is updated as

$$\mathcal{N}_{t+1}^U = \min\{\mathcal{N}_t^Q + \mathcal{N}_t^C, \mathcal{N}_{\max}\}, \quad (9)$$

where $\mathcal{N}_t^C = |\mathcal{J}_t^C|$ is the number of collected tasks from GD $j \in \mathcal{J}_t^C$. The UAV's energy consumption for executing $\min\{\mathcal{N}_t^L, \phi\}$ tasks in t is calculated as

$$E_t^L = \kappa \cdot \min\{\mathcal{N}_t^L, \phi\} \mathcal{IC} \cdot (f_{\text{UAV}})^2, \quad (10)$$

where κ is the effective capacitance coefficient.

On the basis of the Shannon-Hartley theorem [3], the transmission rate from the UAV to the HAP in time slot t is obtained by

$$\varphi_t = \omega \cdot \log_2 \left(1 + \frac{P_{\text{UAV}} \cdot g_t}{\xi^2} \right), \quad (11)$$

where ω is the system bandwidth, and P_{UAV} and ξ^2 are the UAV's transmission and noise powers, respectively. g_t denotes the channel gain between the UAV and HAP in time slot t , which is obtained by the free-space path loss model as follows

$$g_t = g_0 (d_t^{\text{UH}})^{-2} = \frac{g_0}{\|\mathcal{M}_t^U - \mathcal{M}^H\|^2}, \quad (12)$$

where g_0 indicates the channel power gain for the 1 m reference distance, d_t^{UH} is the distance between the UAV and HAP in t , and $\|\cdot\|$ is the euclidean norm.

The UAV needs to transmit the input data of $\mathcal{N}_t^O$ computation tasks to the HAP via a wireless channel. Let η_t represent the elapsed time since the UAV initiated the transmission at the beginning of t and is calculated as

$$\eta_t = \arg\max_l \left(\sum_{i=t}^{t+l} \delta \varphi_i \geq \mathcal{L}_t \right). \quad (13)$$

where $\mathcal{L}_t = \mathcal{IC} \mathcal{N}_t^O$ denotes the total input data size for transmitting $\mathcal{N}_t^O$ tasks. Based on η_t , the transmission delay that the UAV finishes the offloading of $\mathcal{N}_t^O$ tasks is obtained by

$$D_t^O = \begin{cases} (\eta_t - 1)\delta + \frac{\mathcal{L}_t - \sum_{i=t}^{\eta_t-1} \delta \varphi_i}{\varphi_{t+\eta_t}}, & \text{if } \mathcal{L}_t < \sum_{i=t}^{\eta_t+t} \delta \varphi_i \\ \delta \eta_t, & \text{if } \mathcal{L}_t = \sum_{i=t}^{\eta_t+t} \delta \varphi_i \end{cases} \quad (14)$$

where φ_i is the transmission rate in time slot $i (= t, \dots, \eta_t + t)$ and δ is the duration of a time slot. The size of data transmitted by the UAV is $\delta \varphi_i$ within a time slot. Therefore, the total size of data transmitted by the UAV is $\sum_{i=t}^{\eta_t+t} \delta \varphi_i$ after η_t time slots.

The UAV's energy consumption for transmitting these tasks to the HAP is given by

$$E_t^O = P_{\text{UAV}} \cdot D_t^O. \quad (15)$$

We suppose the HAP is equipped with extensive computing resources, the delay incurred in processing offloaded tasks on the HAP can be ignored. Additionally, the delay for returning these task's results to GDs is also neglected since the output data size of a task is generally significantly smaller than its input data size.

D. Problem Formulation

On the basis of (7), in time slot t , we define the MEC system's AoI as the average value of each GD's AoI A_t^j , which is calculated by

$$A_t = \frac{1}{J} \sum_{j=1}^J A_t^j. \quad (16)$$

The total AoI A_{total} of the MEC system during T time slots is calculated by

$$A_{\text{total}} = \sum_{t=1}^T A_t. \quad (17)$$

According to (10) and (15), the UAV's energy consumption (except for propulsion energy consumption) E_t encompasses both the energy consumption generated for local processing E_t^L and energy consumption incurred for task offloading E_t^O . Therefore, the energy consumption E_t is calculated by

$$E_t = E_t^L + E_t^O. \quad (18)$$

The total energy consumption of the UAV during the whole flight mission, E_{total} , is comprised of the execution energy consumption, offloading energy consumption, and propulsion energy consumption. Thus, E_{total} is calculated as

$$E_{\text{total}} = \sum_{t=1}^T E_t + E_{\text{fly}}. \quad (19)$$

Let $\mathbf{U} = \{(b_t, d_t) | \forall t \in \mathcal{T}\}$ and $\mathbf{O} = \{o_t | \forall t \in \mathcal{T}\}$ denote the UAV's flight path and offloading ratio during the whole flight mission, respectively. We formulate the AET problem as an MOP, with A_{total} and E_{total} minimized simultaneously, via

optimizing $\mathbf{U}$ and $\mathbf{O}$. The MOP is defined as

$$\begin{aligned} & \min_{\mathbf{U}, \mathbf{O}} (A_{\text{total}}, E_{\text{total}}) \quad (20a) \\ & \text{subject to:} \\ & 0 \leq b_t \leq 2\pi, \quad \forall t \in \mathcal{T}, \quad (20b) \\ & 0 \leq d_t \leq d_{\max}, \quad \forall t \in \mathcal{T}, \quad (20c) \\ & o_t \in [0, 1], \quad \forall t \in \mathcal{T}, \quad (20d) \\ & 0 \leq X_t^U \leq X_{\max}, \quad \forall t \in \mathcal{T}, \quad (20e) \\ & 0 \leq Y_t^U \leq Y_{\max}, \quad \forall t \in \mathcal{T}, \quad (20f) \\ & d_t^j \leq R_{\max}, \quad \forall j \in \mathcal{J}_t^C, t \in \mathcal{T}, \quad (20g) \\ & A_t^j \leq A_{\max}, \quad \forall j \in \mathcal{J}, t \in \mathcal{T}. \quad (20h) \end{aligned}$$

Inequalities (20b) and (20c) constrain the UAV's flight direction and distance in each time slot. Eq (20d) specifies the task offloading ratio is a variable in the range of 0 to 1. Inequalities (20e) and (20f) together stipulate that the UAV's flight region can not exceed the rectangle's boundary. Inequality (20g) specifies that the UAV is only capable of collecting tasks from GDs within the communication coverage. Inequality (20h) specifies that each GD's AoI can not exceed the maximal AoI $A_{\max}$ that the MEC system can tolerate. One can understand that to maximize A_{total} , the UAV should follow an appropriate path during its flight mission so that it can cover as many GDs and collect their tasks as possible. A positive correlation exists between the number of tasks collected and the AoI reduction. In simple terms, collecting a greater number of tasks from GDs leads to a more rapid decrease in their AoI. Nevertheless, the UAV has to process these collected tasks, incurring its high energy consumption. Hence, it is easily understood that two objectives, i.e., the minimization of A_{total} and minimization of E_{total} , are inherently conflicting with each other.

III. OVERVIEW OF MOMDP AND MOP

A. MOMDP

An MOMDP comprises the state space $\mathcal{S}$, action space $\mathcal{A}$, vectorial reward function $\mathbf{r}$, initial state distribution $\mathcal{U}$, and discount factor $\gamma \in [0, 1]$. A policy $\pi : \mathcal{S} \rightarrow \mathcal{A}$ is a state-to-action mapping which corresponds to an expected vectorial return $\mathbf{R}_\pi = (R_\pi^1, \dots, R_\pi^m)$, where m is the number of objectives. The k -th expected return R_π^k in $\mathbf{R}_\pi$ is associated with the k -th optimization objective of an MOP. R_π^k is defined as

$$R_\pi^k = \mathbb{E}_\pi \left[\sum_{t=1}^T \gamma^{t-1} r_t^k(s_t, a_t) | s_1 \sim \mathcal{U}, a_t \sim \pi(s_t) \right], \quad (21)$$

where r_t^k is the k -th element of the vectorial reward $\mathbf{r}_t = (r_t^1, \dots, r_t^m)$ received at time step t . $s_t \in \mathcal{S}$ and $a_t \in \mathcal{A}$ are the state and action at time step t , respectively. The AET problem has two optimization objectives A_{total} and E_{total} , thus we have $m = 2$. In other words, R_π^1 and R_π^2 correspond to objectives A_{total} and E_{total} , respectively.

B. MOP

An MOP can be formulated as

$$\begin{cases} \min & \mathbf{F}(\pi) = (f^1(\pi), \dots, f^m(\pi)), \\ \text{s.t.} & \pi \in \Pi. \end{cases} \quad (22)$$

where π is a policy in search space Π . $\mathbf{F}(\pi)$ is the objective vector including m objective functions which generally conflict with each other. The AET problem is an MOP that can be modeled by MOMDP. That is to say, we need to set the objective value $f^k(\pi)$ to the expected return R_π^k , $k = 1, \dots, m$.

Let π_1 and π_2 represent two different policies in Π . π_1 is said to dominate π_2 if and only if $f^k(\pi_1) \leq f^k(\pi_2)$ for all $k = 1, \dots, m$, and $f^l(\pi_1) < f^l(\pi_2)$ for at least one index $l \in \{1, \dots, m\}$. If there is no one policy in Π dominating the policy π^* , it is Pareto optimal. All Pareto optimal policies, also referred to as nondominated policies, collectively form a Pareto optimal set, and their projection onto the objective space is recognized as the Pareto front.

In general, two primary approaches exist for addressing an MOP. The first method is to convert the MOP into an SOP via the weighted sum. Although the method has simple mathematical models and high computational efficiency, it can only obtain one compromised policy in a single run. The single policy fails to capture the inherent balance between objectives. In simpler terms, the resulting policy is optimal only for the specific weight configuration (i.e., preference) used. The other approach to address MOPs is to employ multi-objective algorithms [3], [36]. These approaches enable the acquisition of numerous nondominated policies in a single run, thereby illustrating the Pareto-dominance relationship between them, which is crucial information for decision-makers. Despite potential fluctuations in user preferences, the obtained nondominated policies remain in effect. Therefore, the ultimate aim of addressing an MOP is to acquire numerous excellent nondominated policies, where each policy caters to a specific preference. By exploring this set, the decision-makers can identify the policy that best aligns with their current preference, thereby better balancing multiple objectives.

IV. THE PROPOSED ALGORITHM

A. MOMDP Model

To tackle the AET problem using an MORL algorithm, it is imperative to establish a dedicated MOMDP model tailored to the problem at hand. This involves systematically defining the state space, action space, and reward function.

1) *State Space*: A state space is the set of possible states in which the MEC environment can be. The state space is defined as

$$\mathcal{S} = \{s_t | s_t = (d_t^1, \dots, d_t^J, A_t^1, \dots, A_t^J, s_t^U), \forall t \in \mathcal{T}\}, \quad (23)$$

where d_t^j is the horizontal distance between the UAV and GD $j \in \mathcal{J}$ in t and A_t^j is the AoI of GD $j \in \mathcal{J}$ in t . The position and AoI data of all GDs can guide the UAV to follow an appropriate path to collect tasks from GDs and reduce their AoI. $s_t^U = (t, \mathbb{1}_{t-1}, \mathcal{M}_t^U, C_t^U, \mathcal{N}_t^U, \mathcal{N}_t^C)$ is the state of the UAV in t ,

where t indicates that the current state is in the t -th time slot; $\mathbb{1}_{t-1}$ is an indicator variable that is equal to one if the UAV moves outside of the restricted region in $t - 1$, otherwise it is equivalent to zero; $\mathcal{M}_t^U$ is the UAV's coordinate; C_t^U represents the number of times that the UAV moves outside of the rectangular region so far. The first four elements in s_t^U can help to keep the UAV from flying out of the restricted region, avoiding unnecessary waste of resources. $\mathcal{N}_t^U$ and $\mathcal{N}_t^C$ are the numbers of unfinished tasks and collected tasks in the computing queue, respectively. The latter two elements help to guide the UAV to make proper task offloading decisions.

2) *Action Space*: The AET problem aims at optimizing the UAV's flight path and task offloading ratios to minimize A_{total} and E_{total} . Thus, the action space is defined as

$$\mathcal{A} = \{a_t | a_t = (b_t, d_t, o_t), \forall t \in \mathcal{T}\}, \quad (24)$$

where b_t and d_t indicate the horizontal flight direction and distance, respectively, and o_t represents the task offloading ratio. Assume that the UAV can select one of eight directions to move at its current location in each time slot. We have $b_t \in \mathcal{B} = \{0, \frac{\pi}{4}, \frac{2\pi}{4}, \dots, \frac{7\pi}{4}\}$, where $b_t = 0$ denotes the east, $b_t = \frac{\pi}{4}$ indicates the northeast, and $b_t = \frac{2\pi}{4}$ represents the north, etc. Similarly, the continue values d_t and o_t are discretized into seven actions and eleven actions, respectively. Thus, we have $d_t \in \mathcal{D} = \{0, \frac{d_{\max}}{6}, \frac{2d_{\max}}{6}, \dots, d_{\max}\}$ and $o_t \in \mathcal{O} = \{0, 0.1, 0.2, \dots, 1\}$.

3) *Reward Function*: The AET problem aims to minimize A_{total} and E_{total} , simultaneously. As a result, after the agent takes action a_t , the received vectorial reward is defined as

$$\mathbf{r}_t = (r_t^A, r_t^E) = \begin{cases} (\Delta_t^A, -E_t'), & \text{if } \mathbb{1}_t = 0 \\ (\Delta_t^A - \varepsilon_p, -E_t' - \varepsilon_p), & \text{if } \mathbb{1}_t = 1 \end{cases} \quad (25)$$

where r_t^A and r_t^E are two scalar rewards that are associated with A_{total} and E_{total} , respectively. To minimize A_{total} , we set r_t^A to $\Delta_t^A = A_{t-1} - A_t$ that indicates the negative increment of the AoI A_{t-1} after finishing the task collection process in time slot t . We give a positive reward to the agent for its smaller AoI and punish it for its larger AoI. To minimize E_{total} , we set r_t^E to $-E_t' = -(E_t + \varepsilon_b)$, where ε_b is a scaling factor used to ensure that r_t^A and r_t^E have the same order of magnitude. This can enable the simultaneous optimization of two objectives without introducing any biases, thus obtaining a good balance between them. Based on the positive or negative results of E_t , the agent can know how good or how bad the current action decision is, adjusting its actions accordingly in the subsequent time slots. The penalty factor ε_p is adopted to reduce r_t^A and r_t^E if the UAV moves outside of the restricted region in t .

B. MOL-AET Algorithm

A learning individual can be represented as a three-tuple $\Upsilon = \langle \mathbf{w}, \pi_\theta, v_\phi \rangle$, where

- $\mathbf{w} = (w^1, \dots, w^m)$: The weight vector used to denote a user preference between m objectives, satisfying $\sum_{k=1}^m w^k = 1$.
- π_θ : The policy network, parameterized by θ , which interacts with the MEC environment to determine actions based on the current state.

- v_ϕ : The value network, parameterized by ϕ , which evaluates the value of the environmental states observed by the agent.

The framework of MOL-AET is shown in Fig. 3, comprising the initialization phase, training phase, and evolutionary phase. During the initialization phase, n learning individuals are generated, each of which includes a weight vector, a policy network, and a value network. In the training phase, each learning individual is trained using proximal policy optimization (PPO) through interaction with the MEC system environment. A set of nondominated policies are obtained at the end of the training phase. Subsequently, in the evolutionary phase, the genetic operators are employed to further enhance the quality of the nondominated policy set. Next, we explain the reason for launching the evolutionary phase after the training phase.

RL algorithms have been successfully applied to diverse domains, such as UAV path planning [10] and intelligent transportation systems [37]. However, the parameter updating of RL algorithms heavily depends on gradient descent, making it easy to fall into local optimum. Evolutionary algorithms (EAs), characterized by gradient-free, belong to a category of black-box optimization methods inspired by principles of natural evolution [38]. EAs maintain a population of learning individuals and iteratively search for satisfying policies. Thus, EAs have good global exploration abilities and stable convergence. However, they suffer from low sample efficiency and struggle to address high-dimensional problems, such as the modeled AET problem. Therefore, we integrate the heterogeneous policy optimization methods of RL and EA, because both of them have distinct and complementary advantages. In the training phase, we first train each learning individual by PPO. This phase can take full advantage of PPO's local exploitation ability to obtain good optimization results. Then, the evolutionary phase adopts the genetic operators to further improve each learning individual. To be specific, uniform crossover and Gaussian mutation directly operate at the parameter level of a learning individual's policy network. The two operators can enhance the MOL-AET's global exploration ability, avoiding stagnation and premature convergence.

Algorithm 1 shows the pseudo-code of MOL-AET. We elaborate on its three phases in detail.

1) *Initialization Phase*: To simultaneously optimize two objectives A_{total} and E_{total} , MOL-AET maintains n learning individuals which have different weight vectors. Note that each weight vector represents a preference between objectives, meaning these learning individuals can simultaneously optimize them from the perspective of different preferences.

Similar to [3], we employ the systematic approach to produce n uniformly distributed weight vectors, denoted by the set $\mathcal{W} = \{\mathbf{w}_1, \dots, \mathbf{w}_n\}$ (step 2 of Algorithm 1). There are two important parameters m and β in the approach, where m and β represent the numbers of objectives and divisions. Based on the two parameters, the systematic approach can generate $n = C_{m+\beta-1}^{m-1}$ evenly distributed weight vectors. For example, we have $m = 2$ for the AET problem with two objectives. If $\beta = 29$, thirty weight vectors can be generated. The distribution of weight vectors has a great effect on the action schemes of learning individuals. These uniformly distributed weight vectors

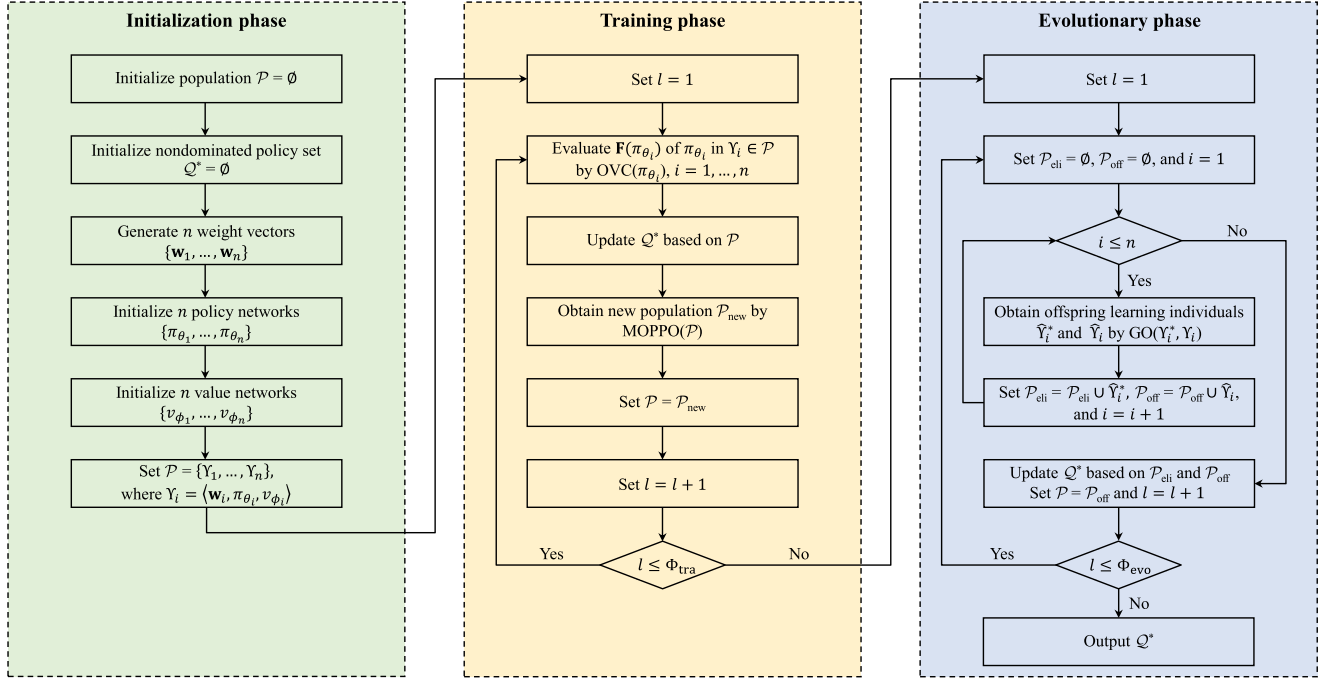


Fig. 3. Framework of MOL-AET.

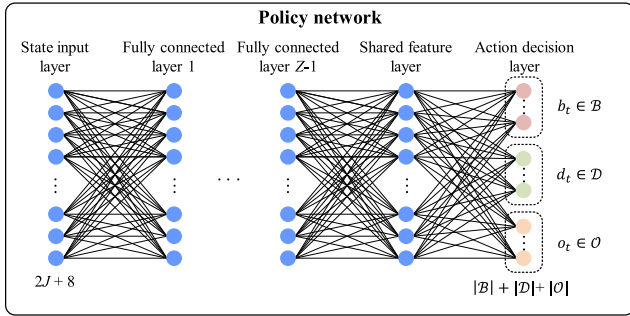


Fig. 4. Policy network architecture.

can guide learning individuals to obtain various nondominated policies. In steps 3 and 4, we randomly initialize n policy networks, $\{\pi_{\theta_1}, \dots, \pi_{\theta_n}\}$, and n value networks, $\{v_{\phi_1}, \dots, v_{\phi_n}\}$. We represent the population $\mathcal{P} = \{\Upsilon_1, \dots, \Upsilon_n\}$, where $\Upsilon_i = \langle \mathbf{w}_i, \pi_{\theta_i}, v_{\phi_i} \rangle$ is the i -th learning individual. As described in Section IV-A2, the action decision a_t consists of the flight direction b_t , flight distance d_t , and offloading ratio o_t , which have $|\mathcal{B}|$, $|\mathcal{D}|$, and $|\mathcal{O}|$ discrete values, respectively. The traditional neural network architecture needs to build a neuron in its output layer for each alternative action. Thus, the number of neurons in the output layer is $|\mathcal{A}| = |\mathcal{B}| \times |\mathcal{D}| \times |\mathcal{O}| = 616$. This may result in the curse of dimensionality, making it difficult to obtain optimal action policies. To overcome the challenge, this paper adopts three subnetworks to establish the output layer of the policy networks. The policy network architecture is shown in Fig. 4, which includes a state input layer, Z fully connected layers, and an output layer (i.e., action decision layer). The state s_t undergoes normalization prior to being fed into the input layer of the policy network. This pre-processing step is necessary

because the state's elements have disparate value ranges. Each subnetwork is corresponding to an action decision variables. The action decision a_t consists of the flight direction b_t , flight distance d_t , and offloading ratio o_t , which have $|\mathcal{B}|$, $|\mathcal{D}|$, and $|\mathcal{O}|$ discrete values, respectively. Thus, the numbers of neurons of three subnetworks are equal to $|\mathcal{B}|$, $|\mathcal{D}|$, and $|\mathcal{O}|$, respectively. The Z -th fully connected layer is the shared feature layer which separately connects each subnetwork.

2) *Training Phase*: In this phase, MOL-AET starts training each learning individual in the population $\mathcal{P}$ by multi-objective proximal policy optimization (MOPPO). First, we adopt Algorithm 2 to evaluate $\mathbf{F}(\pi_{\theta_i})$ of the policy network π_{θ_i} in the learning individual $\Upsilon_i \in \mathcal{P}$, $i = 1, \dots, n$. Then, the nondominated policy set $\mathcal{Q}^*$ is updated based on $\mathcal{P}$. To be specific, we remove any policy dominated by π_{θ_i} from $\mathcal{Q}^*$, and add π_{θ_i} to $\mathcal{Q}^*$ if it is not dominated by any existing policy in $\mathcal{Q}^*$. By doing so, all policies in $\mathcal{Q}^*$ are Pareto optimal.

To train each learning individual $\Upsilon_i \in \mathcal{P}$, a trajectory including multiple transitions need to be collected by interacting with the MEC system environment (steps 3-10 of Algorithm 3). Before inputting the policy network π_{θ_i} , the observed state s_t should be normalized because of different dimensionality. Then, the normalized outcomes are fed into the policy network and it outputs action $a_t = (b_t, d_t, o_t)$. The learning individual Υ_i executes a_t to transition the UAV from its current position to the next position based on the observation of the next state s_{t+1} . Meanwhile, it obtains a vectorial reward $\mathbf{r}_t$ from the environment. The transition $(s_t, a_t, \mathbf{w}_i \mathbf{r}_t, s_{t+1})$ is added to the trajectory Γ_i , where $\mathbf{w}_i \mathbf{r}_t$ is the dot product of $\mathbf{w}_i$ and $\mathbf{r}_t$. Repeat the process above until the maximal time step T is reached.

This paper employs MOPPO to train each police network and value network. To prevent excessive updates to the policy network and ensure stable learning, a clipped surrogate objective

Algorithm 1: MOL-AET.

Input: number of learning individuals n , number of maximal training times Φ_{tra} , number of maximal evolution generations Φ_{evo} .
// Initialization phase
1: Initialize population $\mathcal{P} = \emptyset$ and nondominated policy set $\mathcal{Q}^* = \emptyset$;
2: Generate n weight vectors $\mathcal{W} = \{\mathbf{w}_1, \dots, \mathbf{w}_n\}$;
3: Initialize n policy networks $\{\pi_{\theta_1}, \dots, \pi_{\theta_n}\}$;
4: Initialize n value networks $\{v_{\phi_1}, \dots, v_{\phi_n}\}$;
5: Set $\mathcal{P} = \{\Upsilon_1, \dots, \Upsilon_n\}$, where $\Upsilon_i = \langle \mathbf{w}_i, \pi_{\theta_i}, v_{\phi_i} \rangle$;
// Training phase
6: **for** $l = 1, \dots, \Phi_{\text{tra}}$ **do**
7: Evaluate objective vector $\mathbf{F}(\pi_{\theta_i})$ of policy network π_{θ_i} in Υ_i by OVC(π_{θ_i}), $i = 1, \dots, n$;
8: Update $\mathcal{Q}^*$ based on $\mathcal{P}$;
9: Obtain new population $\mathcal{P}_{\text{new}}$ by MOPPO($\mathcal{P}$);
10: Set $\mathcal{P} = \mathcal{P}_{\text{new}}$;
11: **end for**
// Evolutionary phase
12: **for** $l = 1, \dots, \Phi_{\text{evo}}$ **do**
13: Initialize elite population $\mathcal{P}_{\text{eli}} = \emptyset$ and offspring population $\mathcal{P}_{\text{off}} = \emptyset$;
14: **for** $\Upsilon_i \in \mathcal{P}$ **do**
15: Obtain matched individual Υ_i^* of Υ_i from $\mathcal{Q}^*$;
16: Obtain offspring learning individuals $\hat{\Upsilon}_i^*$ and $\hat{\Upsilon}_i$ by performing genetic operator in GO(Υ_i^*, Υ_i);
17: Set $\mathcal{P}_{\text{eli}} = \mathcal{P}_{\text{eli}} \cup \hat{\Upsilon}_i^*$ and $\mathcal{P}_{\text{off}} = \mathcal{P}_{\text{off}} \cup \hat{\Upsilon}_i$;
18: **end for**
19: Update $\mathcal{Q}^*$ based on $\mathcal{P}_{\text{eli}}$ and $\mathcal{P}_{\text{off}}$;
20: Set $\mathcal{P} = \mathcal{P}_{\text{off}}$;
21: **end for**
Output: nondominated policy set $\mathcal{Q}^*$.

Algorithm 2: Objective Vector Evaluation (OVC).

Input: policy network π_{θ_i} .
1: Initialize objective vector $\mathbf{F}(\pi_{\theta_i}) = (0, \dots, 0)$;
2: Observe initial state s_1 ;
3: **for** $t = 1, \dots, T$ **do**
4: Select action a_t by policy network $\pi_{\theta_i}(s_t)$;
5: Take action a_t and move the UAV to next location;
6: Receive reward $\mathbf{r}_t$ and observe new state s_{t+1} ;
7: Calculate $\mathbf{F}(\pi_{\theta_i}) = \mathbf{F}(\pi_{\theta_i}) + \mathbf{r}_t$;
8: **end for**
Output: objective vector $\mathbf{F}(\pi_{\theta_i})$.

function is implemented, as follows

$$\mathcal{L}_P(\mathbf{w}_i, \theta_i) = \mathbb{E} \left[\sum_{t=1}^T \min \left(\frac{\pi_{\theta_i}(a_t|s_t)}{\pi_{\theta_i^{\text{old}}}(a_t|s_t)} G_t^{\mathbf{w}_i}, \text{clip}_{1-\alpha} \left(\frac{\pi_{\theta_i}(a_t|s_t)}{\pi_{\theta_i^{\text{old}}}(a_t|s_t)} \right) G_t^{\mathbf{w}_i} \right) \right], \quad (26)$$

Algorithm 3: Multi-Objective PPO (MOPPO).

Input: population $\mathcal{P}$.
1: Initialize the new population $\mathcal{P}_{\text{new}} = \emptyset$;
2: **for** $\Upsilon_i = \langle \mathbf{w}_i, \pi_{\theta_i}, v_{\phi_i} \rangle \in \mathcal{P}$ **do**
3: Initialize the trajectory $\Gamma_i = \emptyset$;
4: Observe initial state s_1 ;
5: **for** $t = 1, \dots, T$ **do**
6: Select action a_t by policy network $\pi_{\theta_i}(s_t)$;
7: Take action a_t and move the UAV to next location;
8: Receive reward $\mathbf{r}_t$ and observe new state s_{t+1} ;
9: Add transition $(s_t, a_t, \mathbf{w}_i \mathbf{r}_t, s_{t+1})$ to Γ_i ;
10: **end for**
11: Update π_{θ_i} by (26) using the trajectory Γ_i ;
12: Update v_{ϕ_i} by (28) using the trajectory Γ_i ;
13: Set $\mathcal{P}_{\text{new}} = \mathcal{P}_{\text{new}} \cup \Upsilon_i$;
14: **end for**
Output: new population $\mathcal{P}_{\text{new}}$.

where $\pi_{\theta_i^{\text{old}}}$ is the old policy network before updating π_{θ_i} . $\text{clip}_{1-\alpha}^{\Lambda}(\Lambda)$ is the clip function that limits the value of Λ . $G_t^{\mathbf{w}_i}$ is the general advantage estimator (GAE) [39], which is defined as

$$G_t^{\mathbf{w}_i} = \sum_{l=0}^{T-t+1} (\gamma \lambda)^l (\mathbf{w}_i \mathbf{r}_{t+l} + \gamma v_{\phi_i}(s_{t+l+1}) - v_{\phi_i}(s_{t+l})), \quad (27)$$

where $\mathbf{w}_i \mathbf{r}_{t+l}$ is the dot product of $\mathbf{w}_i$ and $\mathbf{r}_{t+l}$.

The loss of the value function is defined as

$$\mathcal{L}_V(\mathbf{w}_i, \phi_i) = \mathbb{E} \left[\sum_{t=1}^T (v_{\phi_i}(s_t) - (\mathbf{w}_i \mathbf{r}_t + \gamma v_{\phi_i}(s_{t+1})))^2 \right], \quad (28)$$

Based on (26) and (28), two networks π_{θ_i} and v_{ϕ_i} are updated (steps 11 and 12 of Algorithm 3). The trained learning individual Υ_i is added to the new population $\mathcal{P}_{\text{new}}$. In step 9 of Algorithm 3, the new population $\mathcal{P}_{\text{new}}$ is obtained and it is regarded as the next training's population.

The training phase can offer multiple promising learning individuals, each of which resides in a high-performance area of the search space. Leveraging these individuals as the initial population, the MOL-AET's evolutionary procedure is of low noise, thus more possibly to obtain favorable multi-objective optimization outcomes.

3) *Evolutionary Phase:* The evolutionary phase maintains an elite population $\mathcal{P}_{\text{eli}}$ and an offspring population $\mathcal{P}_{\text{off}}$, to further update the nondominated policy set $\mathcal{Q}^*$. In steps 14-18, MOL-AET performs the genetic operator on each learning individual $\Upsilon_i \in \mathcal{P}$ and its matched individual $\Upsilon_i^* \in \mathcal{Q}^*$. For each Υ_i in $\mathcal{P}$, we first compute the euclidean distances between its weight vector $\mathbf{w}_i$ and the weight vectors of all learning individuals in $\mathcal{Q}^*$. Then, that individual in $\mathcal{Q}^*$ with the smallest distance is regarded as the matched individual of Υ_i . We adopt Algorithm 4 to perform the genetic operator on two parent learning individuals, where $\text{rand}(0,1)$ indicates a random number within

Algorithm 4: Genetic Operator (GO).

Input: two parent learning individuals $\Upsilon_i = \langle \mathbf{w}_i, \pi_{\theta_i}, v_{\phi_i} \rangle$ and $\Upsilon_i^* = \langle \mathbf{w}_i^*, \pi_{\theta_i^*}, v_{\phi_i^*} \rangle$.
// Crossover operator
1: **for** $l = 1, \dots, N_{\text{gen}}$ **do**
2: **if** $\text{rand}(0,1) < P_{\text{cro}}$ **then**
3: Set $\theta_{i,l} = \theta_{i,l}^*$;
4: **else**
5: Set $\theta_{i,l}^* = \theta_{i,l}$;
6: **end if**
7: **end for**
// Mutation operator
8: **for** $l = 1, \dots, N_{\text{gen}}$ **do**
9: **if** $\text{rand}(0,1) < P_{\text{mut}}$ **then**
10: **if** $\text{rand}(0,1) < 0.05$ **then**
11: Set $\theta_{i,l} = \mathcal{G}(0, 100 * M_{\text{str}})$ and $\theta_{i,l}^* = \mathcal{G}(0, 100 * M_{\text{str}})$;
12: **else if** $\text{rand}(0,1) < 0.1$ **then**
13: Set $\theta_{i,l} = \mathcal{G}(0, 1)$ and $\theta_{i,l}^* = \mathcal{G}(0, 1)$;
14: **else**
15: Set $\theta_{i,l} = \mathcal{G}(0, M_{\text{str}})$ and $\theta_{i,l}^* = \mathcal{G}(0, M_{\text{str}})$;
16: **end if**
17: **end if**
18: **end for**
19: Set $\hat{\Upsilon}_i = \Upsilon_i$ and $\hat{\Upsilon}_i^* = \Upsilon_i^*$;
Output: offspring learning individuals $\hat{\Upsilon}_i$ and $\hat{\Upsilon}_i^*$.

the range of (0,1). The genetic operator consists of the crossover and mutation operators.

For pair Υ_i and Υ_i^* , the crossover operator is applied to the corresponding policy networks, i.e., π_{θ_i} and $\pi_{\theta_i^*}$. Let N_{gen} denote the number of parameters for each policy network. Let $\theta_{i,l}$ and $\theta_{i,l}^*$ denote the l -th parameter of π_{θ_i} and $\pi_{\theta_i^*}$, respectively. For each parameter $\theta_{i,l}$ (or $\theta_{i,l}^*$), there is a probability P_{cro} that $\theta_{i,l}$ (or $\theta_{i,l}^*$) is replaced with the parameter $\theta_{i,l}^*$ (or $\theta_{i,l}$). Specifically, we first randomly generate a number $\text{rand}(0,1)$ in the range of (0,1). If $\text{rand}(0,1) < P_{\text{cro}}$, the parameter $\theta_{i,l}$ is replaced with the parameter $\theta_{i,l}^*$, otherwise $\theta_{i,l}^*$ is set to $\theta_{i,l}$. In other words, after the crossover operator, a portion of parameters of the policy network π_{θ_i} are replaced with that of the policy network $\pi_{\theta_i^*}$, so does $\pi_{\theta_i^*}$.

The mutation operator plays a significant role in enhancing diversity in evolution. Similar to [38], we also adopt the Gaussian distribution to mutate the parameters of networks. The mutation operator is also applied to the policy networks (steps 8-18 of Algorithm 4). Let M_{str} and $\mathcal{G}(0, \sigma)$ be the mutation strength and Gaussian distribution with the mean 0 and standard deviation σ , respectively. For example, for the policy network π_{θ_i} , we perform mutation on its each parameter $\theta_{i,l}$ with probability P_{mut} . If a random number $\text{rand}(0,1)$ is smaller than the mutation probability P_{mut} , the parameter $\theta_{i,l}$ is changed by performing a drastic Gaussian perturbation (step 11), resetting the parameter (step 13), or a minor Gaussian perturbation (step 15), with probabilities 5%, 5%, and 90%, respectively.

At the end of the genetic operator, the two offspring learning individuals, $\hat{\Upsilon}_i^*$ and $\hat{\Upsilon}_i$, are generated. $\hat{\Upsilon}_i^*$ and $\hat{\Upsilon}_i$ are added to $\mathcal{P}_{\text{eli}}$ and $\mathcal{P}_{\text{off}}$, respectively (step 17). The nondominated policy set $\mathcal{Q}^*$ is updated based on $\mathcal{P}_{\text{eli}}$ and $\mathcal{P}_{\text{off}}$, thereby further improving its quality.

4) *Complexity Analysis:* MOL-AET consists of the initialization phase, training phase, and evolutionary phase. Compared with the training and evolutionary phases, the time complexity (TC) of the initialization phase is trivial and can be neglected. We first analyze the TC of the training phase that has a "for" loop. The loop's TC mainly relies on the generation of the new population $\mathcal{P}_{\text{new}}$ (i.e., step 9 in Algorithm 1), and thus the other steps (i.e., steps 7,8,10 in Algorithm 1) can be neglected. As shown in Algorithm 3, MOPPO optimizes each learning individual in $\mathcal{P}$ to generate a new population $\mathcal{P}_{\text{new}}$, and its TC depends on the training of policy networks. Let Φ_z indicate the number of neurons in the z -th layer, and Φ_0 and Φ_{Z+1} denote the numbers of neurons in the input and output layers, respectively. Hence, the MOPPO's TC is $O(n \times T \times (\sum_{z=1}^{Z+1} \Phi_{z-1} \times \Phi_z))$.

Then, we analyze the TC of the evolutionary phase that has two 'for' loops. The outer 'for' loop is that the population $\mathcal{P}$ evolves Φ_{evo} generations. The TC of the inner loop primarily depends on the genetic operator on two learning individuals (i.e., step 16 in Algorithm 1). According to Algorithm 4, the genetic operator includes the crossover and mutation operators, and their TCs are $O(N_{\text{gen}})$. Note that N_{gen} is the number of parameters for each policy network. Based on $O(N_{\text{gen}})$, the TC of the inner loop is $O(n \times N_{\text{gen}})$. Therefore, the evolutionary phase's TC is $O(\Phi_{\text{evo}} \times n \times N_{\text{gen}})$.

Based on the TCs of the training and evolutionary phases, MOL-AET's TC is $O(n \times (T \times (\sum_{z=1}^{Z+1} \Phi_{z-1} \times \Phi_z) + \Phi_{\text{evo}} \times N_{\text{gen}}))$.

V. SIMULATION RESULT AND ANALYSIS

A. Experiment Setup

To assess the optimization performance of MOL-AET, we implement comprehensive experiments using a custom Python simulator built on PyTorch 1.3. This simulator enables us to comprehensively evaluate and analyze the performance in diverse experimental scenarios. In our experimental setup, we consider a square region and its side lengths, X_{max} and Y_{max} are both restricted to 200 m. Assume that the number of time slots $T = 300$, and each time slot's duration is set to one second [3]. Thus, we have five minutes for the UAV's mission period. A computation task's input data size $\mathcal{I}$ and computation intensity $\mathcal{C}$ are set to 5 MB and 10 cycles/bit, respectively [40].

The number of learning individuals n is configured as 30 for the proposed MOL-AET. Each learning individual corresponds to a weight vector. Thus, we need to generate 30 uniformly distributed weight vectors. For each learning individual, its policy network has three fully connected layers. Each layer comprises 500 neurons and utilizes the rectified linear unit (ReLU) as the activation function. Note that the output layer of the policy network adopts the Softmax activation function to select actions. The value network shares the same architecture as the policy network except for the output layers. The Adam

TABLE III
SIMULATION PARAMETER SETTINGS

Parameter	Value	Parameter	Value
$A_{\max}$	300	ε_p	100
d_0	0.6	η	0.503 m^2
$d_{\max}$	30 m	$\vartheta_{\max}$	$\pi/4$
f_{UAV}	1 GHz	κ	10^{-26} dBm
g_0	10^{-3} GHz	λ	0.95
H	20 km	$\nu_{\max}$	30 m/s
P_{cro}	0.5	ν_{rot}	4.03 m/s
P_{bla}	79.86 W	ν_{tip}	120 m/s
P_{ind}	88.63 W	ξ^2	-174 dBm
P_{UAV}	5 W	ρ	1.225 km/m^3
M_{str}	0.1	Φ_{evo}	100
$N_{\max}$	10	Φ_{tra}	100
α	0.2	ψ	0.05
γ	0.99	ω	10 MHz

TABLE IV
TEST INSTANCES

Instance (J, U)	Number of GDs (J)	Flight height (U)
Ins-(60,30)	60	30
Ins-(60,40)	60	40
Ins-(60,50)	60	50
Ins-(100,30)	100	30
Ins-(100,40)	100	40
Ins-(100,50)	100	50
Ins-(140,30)	140	30
Ins-(140,40)	140	40
Ins-(140,50)	140	50

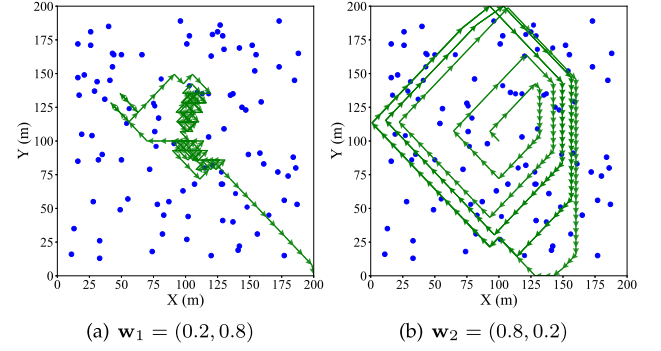
optimizer is employed with a learning rate of 0.0003 for training the neural networks. For the genetic operator, the crossover probability P_{cro} is set to 0.5. The mutation probability P_{mut} for each parameter of a policy network is randomly produced within the range of (0,1) [38]. The related parameters of the UAV's propulsive energy consumption defined in (3) refer to [41]. Table III lists other parameter values used in the experiment.

For the modeled AET problem, there exist no benchmark datasets in the literature. A set of test instances are thus produced to validate MOL-AET's performance. To be specific, we generate various test instances using two significant parameters, including the number of GDs J and the UAV's flight height U . We specify $J \in \{60, 100, 140\}$ and $U \in \{30, 40, 50\}$. On the basis of different combinations between J and U , we can produce nine test instances to simulate different MEC networks, as depicted in Table IV.

Six extensively used evaluation indicators are adopted to measure the performance of MOL-AET, consisting of two algorithm-related indicators (i.e., the inverted generational distance (IGD) [3] and generational distance (GED) [41]), three system-related indicators (i.e., average AoI (AAoI), average energy consumption (AEC), and average cost (AC) [3]), and a non-parametric test (i.e., Friedman test [42]).

B. Performance Evaluation

We utilize Ins-(140,50) in Table IV as a case study to examine the UAV's flight paths. Suppose the decision-maker's current preferences can be indicated by $\mathbf{w}_1 = (0.2, 0.8)$ and $\mathbf{w}_2 = (0.8, 0.2)$. Preference $\mathbf{w}_1$ denotes that the decision-maker should pay more attention to reducing the total energy

Fig. 5. Two flight paths corresponding to $\mathbf{w}_1$ and $\mathbf{w}_2$.

consumption (i.e., E_{total}). Preference $\mathbf{w}_2$ means that one is more concerned with decreasing the total AoI (i.e., A_{total}) than E_{total} .

After running MOL-AET, we obtain multiple nondominated policies (i.e., $\mathcal{Q}^*$), each of which optimization a certain preference. We select two policies from $\mathcal{Q}^*$, which are best matched with $\mathbf{w}_1$ and $\mathbf{w}_2$. Based on the two policies, the UAV's two flight paths are obtained by simple algebraic calculations, as illustrated in Fig. 5, where blue circles denote GDs. Note that the UAV's take-off point is set to the central point. Fig. 5(a) shows the UAV's flight path associated with $\mathbf{w}_1$ that prefers to minimize E_{total} . The UAV moves a short distance in almost all time slots, reducing its propulsion power consumption. Moreover, the UAV can not cover overmuch GDs and collect their tasks, decreasing its processing energy consumption. Fig. 5(b) depicts the flight path corresponding to $\mathbf{w}_2$ that places more focus on the AoI reduction. It is seen that the UAV travels long distances in the rectangle region, thus it can collect more tasks from GDs and decrease their AoIs. But long-distance travel results in higher propulsion power consumption.

In order to fully investigate MOL-AET's performance, we implement five state-of-the-art algorithms for comparison, including two multi-objective evolutionary algorithms (MOEAs), namely NSGA-II and MOEA/D, and three multi-objective reinforcement learning (MORL) algorithms, namely MODQN, MOPG, and MOPPO. NSGA-II and MOEA/D are two widely acknowledged MOEAs in the field of multi-objective optimization. They can obtain multiple nondominated policies after a run, each of which optimizes a certain preference. MODQN is the multi-objective DQN that is a representative RL method among value function based methods. MOPG and MOPPO are policy-based RL methods. The three state-of-the-art RL algorithms have been widely used to address complex control problems. The compared algorithms are briefly described as follows.

- NSGA-II: The nondominated sorting genetic algorithm II is employed to balance the task delay and energy consumption, aiming at meeting user requirements of diverse applications [43].
- MOEA/D: The multi-objective evolutionary algorithm based on decomposition is adopted to decrease the application delay and energy consumption in an MEC system [41].
- MODQN: The multi-objective deep Q-network (DQN), a variant of MOL-AET that adopts DQN [44] instead

TABLE V
IGD VALUES OF SIX ALGORITHMS

Algorithm	Ins-(60,30)	Ins-(60,40)	Ins-(60,50)	Ins-(100,30)	Ins-(100,40)	Ins-(100,50)	Ins-(140,30)	Ins-(140,40)	Ins-(140,50)
NSGA-II	11627.26	11782.18	13220.51	11516.77	12595.62	14424.34	11750.16	12202.07	12338.36
MOEA/D	12337.19	13748.40	14495.31	12560.11	13849.50	14252.87	14314.24	12440.83	11744.59
MODQN	7168.83	3752.79	8543.76	6408.52	6394.54	7556.20	6554.33	4706.70	6619.62
MOPG	4575.39	1086.36	4757.04	2068.15	3469.19	5254.30	4691.42	2433.92	5141.54
MOPPO	2668.02	684.77	2504.46	2980.25	2149.29	4747.21	2660.24	2494.04	3623.75
MOL-AET	0.22	0.00	66.72	61.68	125.96	0.00	4.15	71.94	59.26

TABLE VI
GED VALUES OF SIX ALGORITHMS

Algorithm	Ins-(60,30)	Ins-(60,40)	Ins-(60,50)	Ins-(100,30)	Ins-(100,40)	Ins-(100,50)	Ins-(140,30)	Ins-(140,40)	Ins-(140,50)
NSGA-II	105.28	111.95	112.08	104.89	110.30	105.78	106.93	109.75	104.75
MOEA/D	110.63	114.02	111.43	108.01	108.76	112.16	112.17	109.87	103.32
MODQN	81.53	82.25	82.29	77.89	79.84	88.83	30.00	81.30	86.19
MOPG	48.57	86.62	48.95	38.19	48.20	43.54	60.67	40.45	55.79
MOPPO	28.34	28.55	27.52	24.10	25.45	36.89	44.93	60.94	39.28
MOL-AET	0.00	0.00	7.29	5.99	11.01	9.98	2.00	0.00	0.00

of MOPPO, i.e., Algorithm 3. We design MODQN for performance comparison purpose.

- MOPG: The multi-objective policy gradient (PG), a variant of MOL-AET that employs PG [24] instead of MOPPO, i.e., Algorithm 3. We develop MOPG for performance comparison purpose.
- MOPPO: The multi-objective proximal policy optimization (PPO), a variant of MOL-AET without the evolutionary phase. We create MOPPO for performance comparison purpose.
- MOL-AET: The proposed algorithm.

In the above two MOEAs, the population size and maximal number of evolution generations are set to 30 and 200, respectively. The crossover and mutation probabilities are set to 0.8 and 1/300, respectively. Note that a chromosome denotes a solution to the AET problem, and each gene in the chromosome indicates a path planning and an offloading ratio in a time slot. Thus, the chromosome's coding length is 900, which has a high-dimensional search space. It is quite challenging to find optimal solutions. Besides, MODQN, MOPG, MOPPO, and MOL-AET adopt identical parameter configurations for the purpose of fair comparison.

The results of IGD and GED are illustrated in Tables V and VI. The IGD indicator reflects the diversification and convergence of multiple nondominated policies. It is observed that two widely acknowledged MOEAs, NSGA-II and MOEA/D, exhibit very poor performance, manifesting a failure to obtain satisfactory nondominated policies. This inadequacy can be principally ascribed to the inherent challenges associated with high-dimensional MOPs within dynamic environments, exemplified by the AET problem. MOEAs often expend a considerable amount of time in their pursuit of viable nondominated policies, rendering the achievement of convergence challenging within stringent temporal constraints. Specifically, MOEAs characterized by large encoding lengths (i.e., 900) undergo a time-intensive process in generating acceptable nondominated policies. Moreover, the dynamic and uncertain characteristics of the UAV-assisted MEC environment exacerbate these challenges. These dynamics and uncertainties frequently necessitate the relaunch of MOEAs from the beginning, increasing computational overhead and slowing convergence speed.

Second, in all test instances, both MOL-AET and three MORLs outperform NSGA-II and MOEA/D. MOEAs rely on a single chromosome to make decisions for all time slots, while MORLs engage in real-time decision-making for each time slot on the basis of current environments. Significantly, MORLs integrate RL with deep neural networks, equipping them to navigate sequential decision-making challenges in dynamic MEC environments. This adaptability to environmental changes is realized through iterative interactions with the environment. Consequently, MORLs demonstrate swift convergence and responsiveness to user requirements. These attributes position RL as the best choice for addressing the AET problem in comparison to MOEAs.

Finally, MOPPO performs better than MODQN and MOPG in almost all test instances. This is because MOPPO adopts the clipped surrogate objective and general advantage estimator to train the policy networks, avoiding a large update of the neural networks. This is why we choose PPO to optimize each learning individual. MOL-AET integrates the evolutionary phase into MOPPO, thus it outperforms MOPPO in all test instances. After the training phase, MOL-AET iteratively performs the genetic operator on each learning individual $\Upsilon_i \in \mathcal{P}$ and its matched individual $\Upsilon_i^* \in \mathcal{Q}^*$. The generated offspring learning individuals are adopted to further improve the quality of the nondominated policy set $\mathcal{Q}^*$. Thus, the proposed MOL-AET can obtain the best multi-objective optimization performance in all test instances.

To further substantiate the aforementioned observations, we draw the convergence curves of IGD achieved by all algorithms in Fig. 6. Note that the IGD values of all algorithms are normalized for the purpose of presentation. It is evident that MOL-AET consistently outperforms the other five algorithms in all test instances. Furthermore, after launching the evolutionary iteration in the 100th generation, the MOL-AET's performance is further improved. This demonstrates once again the effectiveness of the evolutionary phase.

Tables VII and VIII collect the results of AAoI and AEC obtained by all algorithms, respectively, with the optimal results highlighted in bold. It is noted that the parameters J and U in Ins- (J, U) represent the number of GDs and the UAV's flight height, respectively. For instance, Ins-(60,30) denotes J and U

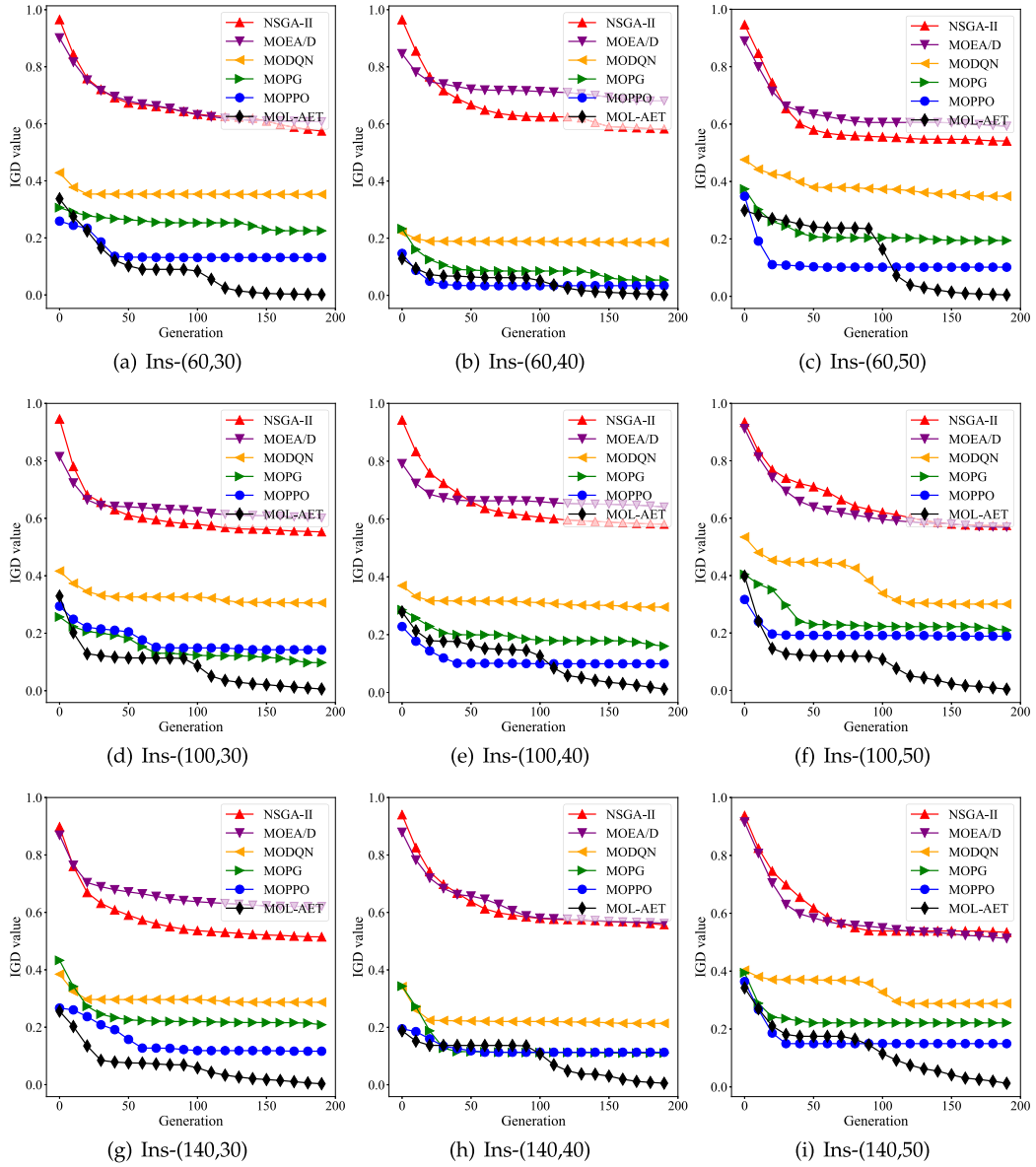


Fig. 6. Convergence curves of six algorithms in terms of IGD.

TABLE VII
AAOI VALUES OF SIX ALGORITHMS

Algorithm	Ins-(60,30)	Ins-(60,40)	Ins-(60,50)	Ins-(100,30)	Ins-(100,40)	Ins-(100,50)	Ins-(140,30)	Ins-(140,40)	Ins-(140,50)
NSGA-II	33811.23	25334.02	20676.00	35751.64	29857.80	30350.71	36048.29	28986.07	21291.87
MOEA/D	32566.29	26041.13	23824.29	34865.15	27203.08	21980.71	32831.62	22843.16	18774.48
MODQN	37139.82	33356.09	26930.04	31422.25	29747.30	22250.62	34555.71	29529.53	25895.61
MOPG	33800.18	26844.56	23654.66	25187.30	29162.18	21739.94	29836.84	24057.69	24873.38
MOPPO	29289.40	23781.18	15911.59	31847.97	20526.60	22224.68	28050.30	24557.37	19090.80
MOL-AET	18683.62	15855.36	10405.48	19675.29	12136.68	9164.67	20216.61	11937.98	11346.65

TABLE VIII
AEC VALUES OF SIX ALGORITHMS

Algorithm	Ins-(60,30)	Ins-(60,40)	Ins-(60,50)	Ins-(100,30)	Ins-(100,40)	Ins-(100,50)	Ins-(140,30)	Ins-(140,40)	Ins-(140,50)
NSGA-II	50613.21	52658.89	53555.05	50950.56	52563.00	52485.80	50663.43	53439.90	53445.53
MOEA/D	51195.70	52090.43	52399.55	50769.44	52032.02	52938.84	52185.68	52898.90	52688.09
MODQN	46129.61	47443.88	50493.29	48630.59	49836.52	50819.30	47443.92	48407.81	50104.89
MOPG	46972.98	43668.32	47027.10	43524.06	47397.17	43756.20	48016.35	42856.68	45638.94
MOPPO	40931.31	41968.26	43105.82	42063.50	42363.10	43596.43	45136.70	45963.32	43864.43
MOL-AET	40952.13	42720.17	43011.35	41971.25	40755.60	42888.46	42366.46	40705.95	45341.62

TABLE IX
AC VALUES OF SIX ALGORITHMS

Algorithm	Ins-(60,30)	Ins-(60,40)	Ins-(60,50)	Ins-(100,30)	Ins-(100,40)	Ins-(100,50)	Ins-(140,30)	Ins-(140,40)	Ins-(140,50)
NSGA-II	40135.17	37050.35	35185.83	41056.82	39689.15	38892.07	41241.95	39247.40	35599.90
MOEA/D	40245.68	38409.00	37612.73	41857.64	39208.82	36803.57	42071.58	36940.09	34920.51
MODQN	37657.73	36240.30	33062.55	34654.75	34628.95	31261.60	36102.22	34297.42	32572.44
MOPG	35666.49	32218.57	31172.48	31281.32	33834.02	30574.21	33585.83	31379.32	31858.63
MOPPO	33317.07	30563.17	26662.47	34776.10	29179.61	30366.48	32826.26	31632.98	29300.16
MOL-AET	28097.25	26806.33	24256.11	28572.23	25251.10	23539.93	28790.80	25171.12	25476.27

TABLE X
AVERAGE RANKS AND POSITIONS OF SIX ALGORITHMS BASED ON IGD, GED, AAOI, AEC, AND AC

Algorithm	IGD		GED		AAOI		AEC		AC	
	Average rank	Position	Average rank	Position	Average rank	Position	Average rank	Position	Average rank	Position
NSGA-II	5.22	5	5.55	6	5.00	5	5.66	6	5.44	5
MOEA/D	5.77	6	5.44	5	3.44	3	5.33	5	5.55	6
MODQN	4.00	4	3.66	4	5.22	6	3.77	4	3.88	4
MOPG	2.77	3	3.11	3	3.55	4	3.11	3	2.77	3
MOPPO	2.22	2	2.22	2	2.77	2	1.77	2	2.33	2
MOL-AET	1.00	1	1.00	1	1.00	1	1.33	1	1.00	1

are 60 and 30, respectively. In Table VII, what is interesting is that the corresponding AAOI values show a tendency to decrease as the parameter U increases when the parameter J is kept constant. This is because the UAV flies at a higher altitude, the more GDs can be covered by the UAV so that it can collect plenty of computation tasks. Thus, the AoI of each GD within the UAV's coverage can be reduced. On the other hand, MOL-AET is the optimal one because it obtains the smallest AAOI values in all test instances. This means MOL-AET can collect computation tasks from GDs as early as possible via properly planning the UAV's flight paths, thereby reducing the AoI of GDs. In addition, the average AoI obtained by MOL-AET can be reduced on average by 50.6%, 46.3%, 52.2%, 45.9%, and 39.9%, compared with NSGA-II, MOEA/D, MODQN, MOPG, and MOPPO, respectively.

As for the results of AEC shown in Table VIII, MOL-AET surpasses the other five algorithms in almost all instances except for Ins-(60,30), Ins-(60,40), and Ins-(140,50), demonstrating its effectiveness in decreasing the energy consumption. Although MOPPO achieves optimal AEC values in the above three instances, the algorithm exhibits inferior performance compared to MOL-AET in terms of AAOI in these instances. For example, while MOPPO has the smallest AEC value (i.e., 40931.31) in Ins-(60,30), it is worse than MOL-AET with respect to AAOI in the instance since MOL-AET obtains the optimal AAOI value (i.e., 18683.62). A similar phenomenon can be observed on MOPPO in Ins-(60,40) and Ins-(140,50). The average energy consumption achieved by MOL-AET can be decreased on average by 19.1%, 18.9%, 13.3%, 6.9%, and 2.1%, compared with NSGA-II, MOEA/D, MODQN, MOPG, and MOPPO, respectively.

Table IX highlights the AC values achieved by six algorithms. Although MOL-AET does not always reach optimal results in all test instances with respect to AEC, it outperforms the other five algorithms regarding AC. Moreover, the average cost obtained by MOL-AET is reduced on average by 32.2%, 32.2%, 24.0%, 19.1%, and 15.3%, compared with NSGA-II, MOEA/D, MODQN, MOPG, and MOPPO, respectively. The AC indicator

can reflect the overall performance of a multi-objective algorithm. Therefore, MOL-AET can strike a tradeoff between the two objectives A_{total} and E_{total} . Based on the results of IGD, GED, AAOI, AEC, and AC, Table X shows the average ranks and positions of six algorithms. It is apparent that MOL-AET achieves optimal comprehensive performance.

VI. CONCLUSION

This paper proposes a novel multi-objective learning algorithm consisting of the training and evolutionary phases, namely MOL-AET, to address the tradeoff between AoI and energy consumption. We first transform the AET problem into an MOMDP by designing the state space, action space, and reward function. MOL-AET obtains a set of excellent nondominated policies, catering to diverse user preferences and achieving a desirable balance between AoI and energy consumption. Compared with NSGA-II, MOEA/D, MODQN, MOPG, and MOPPO, our algorithm achieves better multi-objective optimization performance in all test instances regarding algorithm-related indicators, including the inverted generational distance and generational distance. Compared with the five algorithms, MOL-AET also achieves improvements of at least 39.8%, 2.1%, and 15.3% across several system-related indicators, comprising the average AoI, average energy consumption, and average cost. Moreover, MOL-AET locates the optimal rank in the Friedman test for both algorithm- and system-related indicators. Hence, the performance comparison highlights the excellent performance of MOL-AET and its potential suitability to address the AET problem.

In the future, we will consider a multi-UAV collaborative MEC system, where multiple UAVs are dispatched to collect tasks of GDs and handle them. An MOP problem will be formulated, aiming at minimizing the AoI of GDs and energy consumption of UAVs by jointly optimizing their flight paths and task offloading ratios. Nevertheless, owing to the intricate nature of collision avoidance and collaboration services among UAVs, planning flight paths and determining offloading ratios

for the multiple UAV scenario remains a huge challenge. To tackle this issue, we will develop a multi-agent multi-objective approach incorporating the game theory, where each UAV is treated as an agent. Within this multi-agent system, each UAV takes into account the flying situations of other UAVs to decide their flight paths. This multi-agent approach can reduce computational complexity and prevent collisions among UAVs.

REFERENCES

- [1] P. Mach and Z. Becvar, "Mobile edge computing: A survey on architecture and computation offloading," *IEEE Commun. Surveys Tut.*, vol. 19, no. 3, pp. 1628–1656, Third Quarter, 2017.
- [2] X. Chen, C. Wu, Z. Liu, N. Zhang, and Y. Ji, "Computation offloading in beyond 5G networks: A distributed learning framework and applications," *IEEE Wireless Commun.*, vol. 28, no. 2, pp. 56–62, Apr. 2021.
- [3] F. Song et al., "Evolutionary multi-objective reinforcement learning based trajectory control and task offloading in UAV-assisted mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 12, pp. 7387–7405, Dec. 2023.
- [4] K. Zhang, X. Gui, D. Ren, and D. Li, "Energy-latency tradeoff for computation offloading in UAV-assisted multiaccess edge computing system," *IEEE Internet Things J.*, vol. 8, no. 8, pp. 6709–6719, Apr. 2021.
- [5] Z. Ning et al., "Dynamic computation offloading and server deployment for UAV-enabled multi-access edge computing," *IEEE Trans. Mobile Comput.*, vol. 22, no. 5, pp. 2628–2644, May 2023.
- [6] X. Pang, N. Zhao, J. Tang, C. Wu, D. Niyato, and K.-K. Wong, "IRS-assisted secure UAV transmission via joint trajectory and beamforming design," *IEEE Trans. Commun.*, vol. 70, no. 2, pp. 1140–1152, Feb. 2022.
- [7] Z. Qin et al., "AoI-aware scheduling for air-ground collaborative mobile edge computing," *IEEE Trans. Wireless Commun.*, vol. 22, no. 5, pp. 2989–3005, May 2023.
- [8] L. Liu, K. Xiong, J. Cao, Y. Lu, P. Fan, and K. B. Letaief, "Average AoI minimization in UAV-assisted data collection with RF wireless power transfer: A deep reinforcement learning scheme," *IEEE Internet Things J.*, vol. 9, no. 7, pp. 5216–5228, Apr. 2022.
- [9] T. Liang, W. Liu, J. Yang, and T. Zhang, "Age of information based scheduling for UAV aided emergency communication networks," in *Proc. IEEE Int. Conf. Commun.*, 2022, pp. 5128–5133.
- [10] Y. Liao and V. Friderikos, "Energy and age pareto optimal trajectories in UAV-assisted wireless data collection," *IEEE Trans. Veh. Technol.*, vol. 71, no. 8, pp. 9101–9106, Aug. 2022.
- [11] J. Liu, P. Tong, X. Wang, B. Bai, and H. Dai, "UAV-aided data collection for information freshness in wireless sensor networks," *IEEE Trans. Wireless Commun.*, vol. 20, no. 4, pp. 2368–2382, Apr. 2021.
- [12] Y. Yang et al., "AoI optimization for UAV-aided MEC networks under channel access attacks: A game theoretic viewpoint," in *Proc. IEEE Int. Conf. Commun.*, 2022, pp. 1–6.
- [13] Y. Yang, W. Wang, L. Liu, K. Dev, and N. M. F. Qureshi, "AoI optimization in the UAV-aided traffic monitoring network under attack: A Stackelberg game viewpoint," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 1, pp. 932–941, Jan. 2023.
- [14] Z. Han, Y. Yang, W. Wang, L. Zhou, T. N. Nguyen, and C. Su, "Age efficient optimization in UAV-aided VEC network: A game theory viewpoint," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 12, pp. 25 287–25 296, Dec. 2022.
- [15] Y. Chen, K. Li, Y. Wu, J. Huang, and L. Zhao, "Energy efficient task offloading and resource allocation in air-ground integrated MEC systems: A distributed online approach," *IEEE Trans. Mobile Comput.*, early access, Dec. 25, 2023, doi: [10.1109/TMC.2023.3346431](https://doi.org/10.1109/TMC.2023.3346431).
- [16] Z. Jia, Q. Wu, C. Dong, C. Yuen, and Z. Han, "Hierarchical aerial computing for Internet of Things via cooperation of HAPs and UAVs," *IEEE Internet Things J.*, vol. 10, no. 7, pp. 5676–5688, Apr. 2023.
- [17] E. Eldeeb et al., "A learning-based trajectory planning of multiple UAVs for AoI minimization in IoT networks," in *Proc. IEEE Joint Eur. Conf. Netw. Commun. 6G Summit*, 2022, pp. 172–177.
- [18] B. Choudhury, V. K. Shah, A. Ferdowsi, J. H. Reed, and Y. T. Hou, "AoI-minimizing scheduling in UAV-relayed IoT networks," in *Proc. IEEE Int. Conf. Mobile Ad Hoc Smart Syst.*, 2021, pp. 117–126.
- [19] T. Wu et al., "A novel AI-based framework for AoI-optimal trajectory planning in UAV-assisted wireless sensor networks," *IEEE Trans. Wireless Commun.*, vol. 21, no. 4, pp. 2462–2475, Apr. 2022.
- [20] Q. Dang, Q. Cui, Z. Gong, X. Zhang, X. Huang, and X. Tao, "AoI oriented UAV trajectory planning in wireless powered IoT networks," in *Proc. IEEE Wireless Commun. Netw. Conf.*, 2022, pp. 884–889.
- [21] M. Yi, X. Wang, J. Liu, Y. Zhang, and B. Bai, "Deep reinforcement learning for fresh data collection in UAV-assisted IoT networks," in *Proc. IEEE Conf. Comput. Commun. Workshops*, 2020, pp. 716–721.
- [22] H. Chen, X. Qin, Y. Li, and N. Ma, "Energy-aware path planning for obtaining fresh updates in UAV-IoT MEC systems," in *Proc. IEEE Wireless Commun. Netw. Conf.*, 2022, pp. 1791–1796.
- [23] M. Samir, C. Assi, S. Sharafeddine, and A. Ghayeb, "Online altitude control and scheduling policy for minimizing AoI in UAV-assisted IoT wireless networks," *IEEE Trans. Mobile Comput.*, vol. 21, no. 7, pp. 2493–2505, Jul. 2022.
- [24] B. Zhu, E. Bedeer, H. H. Nguyen, R. Barton, and Z. Gao, "UAV trajectory planning for AoI-minimal data collection in UAV-aided IoT networks by transformer," *IEEE Trans. Wireless Commun.*, vol. 22, no. 2, pp. 1343–1358, Feb. 2023.
- [25] M. Sun, X. Xu, X. Qin, and P. Zhang, "AoI-energy-aware UAV-assisted data collection for IoT networks: A deep reinforcement learning method," *IEEE Internet Things J.*, vol. 8, no. 24, pp. 17 275–17 289, Dec. 2021.
- [26] X. Chen et al., "Information freshness-aware task offloading in air-ground integrated edge computing systems," *IEEE J. Sel. Areas Commun.*, vol. 40, no. 1, pp. 243–258, Jan. 2022.
- [27] F. Wu, H. Zhang, J. Wu, Z. Han, H. V. Poor, and L. Song, "UAV-to-device underlay communications: Age of information minimization by multi-agent deep reinforcement learning," *IEEE Trans. Commun.*, vol. 69, no. 7, pp. 4461–4475, Jul. 2021.
- [28] O. S. Oubbati, M. Atiquzzaman, A. Lakas, A. Baz, H. Alhakami, and W. Alhakami, "Multi-UAV-enabled AoI-aware WPCN: A multi-agent reinforcement learning strategy," in *Proc. IEEE Conf. Comput. Commun. Workshops*, 2021, pp. 1–6.
- [29] Y. Yang, J. Yang, H. Xu, J. Hu, and T. Song, "Trajectory and offloading policy optimization in age-of-information-aware UAV-assisted MEC systems," in *Proc. IEEE Int. Conf. Netw. Appl.*, 2023, pp. 175–180.
- [30] Z. Cheng, M. Liwang, N. Chen, L. Huang, X. Du, and M. Guizani, "Deep reinforcement learning-based joint task and energy offloading in UAV-aided 6G intelligent edge networks," *Comput. Commun.*, vol. 192, pp. 234–244, Jun. 2022.
- [31] H. Kang, X. Chang, J. Mišić, V. B. Mišić, J. Fan, and Y. Liu, "Cooperative UAV resource allocation and task offloading in hierarchical aerial computing systems: A MAPPO based approach," *IEEE Internet Things J.*, vol. 10, no. 12, pp. 10 497–10 509, Jun. 2023.
- [32] L. Li and H. He, "Bipartite graph based multi-view clustering," *IEEE Trans. Knowl. Data Eng.*, vol. 34, no. 7, pp. 3111–3125, Jul. 2022.
- [33] B. Jiang, S. Chen, B. Wang, and B. Luo, "MGLNN: Semi-supervised learning via multiple graph cooperative learning neural networks," *Neural Netw.*, vol. 153, pp. 204–214, 2022.
- [34] A. Singh, K. Raj, T. Kumar, S. Verma, and A. M. Roy, "Deep learning-based cost-effective and responsive robot for autism treatment," *Drones*, vol. 7, no. 2, 2023, Art. no. 81.
- [35] Y. Yu, J. Tang, J. Huang, X. Zhang, D. K. C. So, and K.-K. Wong, "Multi-objective optimization for UAV-assisted wireless powered IoT networks based on extended DDPG Algorithm," *IEEE Trans. Commun.*, vol. 69, no. 9, pp. 6361–6374, Sep. 2021.
- [36] Y. Gong, K. Bian, F. Hao, Y. Sun, and Y. Wu, "Dependent tasks offloading in mobile edge computing: A multi-objective evolutionary optimization strategy," *Future Gener. Comput. Syst.*, vol. 148, pp. 314–325, Nov. 2023.
- [37] X. Liu, C. Sun, K.-L. A. Yau, and C. Wu, "Joint collaborative big spectrum data sensing and reinforcement learning based dynamic spectrum access for cognitive Internet of Vehicles," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 1, pp. 805–815, Jan. 2024.
- [38] S. Khadka and K. Tumer, "Evolution-guided policy gradient in reinforcement learning," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2018, pp. 1–13.
- [39] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," in *Proc. Int. Conf. Learn. Representations*, 2016, pp. 1–14.
- [40] C. Zhou et al., "Deep reinforcement learning for delay-oriented IoT task scheduling in SAGIN," *IEEE Trans. Wireless Commun.*, vol. 20, no. 2, pp. 911–925, Feb. 2021.
- [41] F. Song, H. Xing, S. Luo, D. Zhan, P. Dai, and R. Qu, "A multiobjective computation offloading algorithm for mobile-edge computing," *IEEE Internet Things J.*, vol. 7, no. 9, pp. 8780–8799, Sep. 2020.

- [42] X. Wang, H. Xing, F. Song, S. Luo, P. Dai, and B. Zhao, "On jointly optimizing partial offloading and SFC mapping: A cooperative dual-agent deep reinforcement learning approach," *IEEE Trans. Parallel Distrib. Syst.*, vol. 34, no. 8, pp. 2479–2497, Aug. 2023.
- [43] L. Cui et al., "Joint optimization of energy consumption and latency in mobile edge computing for Internet of Things," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4791–4803, Jun. 2019.
- [44] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, Feb. 2015.



Fuhong Song received the MEng degree in computer technology and the PhD degree in computer science and technology from Southwest Jiaotong University, Chengdu, China, in 2018 and 2022, respectively. He is currently a lecturer with the School of Information, Guizhou University of Finance and Economics. His research interests include mobile edge computing, multi-objective optimization, and reinforcement learning.



Qixun Yang is currently working toward the MS degree in computer science and technology with the Guizhou University of Finance and Economics, China. His current research interests include deep reinforcement learning, edge computing, multi-objective optimization, and unmanned aerial vehicles.



Mingsen Deng (Member, IEEE) is a professor of computer science with the School of Information, Guizhou University of Finance and Economics in China. He has published more the 100 papers in prestigious journals and distinguished international conferences. He has been an executive member of Technical Committee of High-Performance Computing of China Computer Federation since 2010, and received numerous awards, including Outstanding Scientists and Technologists of the Chinese Institute of Electronics (2020), One Hundred Person Project of the Guizhou Province (2016), Young Scientist Award of Guizhou Province (2018). His research interests focus on parallel and distributed computing, electronic structure calculations, and network analysis for big-data.



Huanlai Xing (Member, IEEE) received the PhD degree in computer science from the University of Nottingham (Supervisor: Dr Rong Qu), Nottingham, U.K., in 2013. He was a visiting scholar in computer science with the University of Rhode Island (Supervisor: Dr. Haibo He), USA, in 2020–2021. He is with the School of Computing and Artificial Intelligence, Southwest Jiaotong University (SWJTU), and Tangshan Institute of SWJTU. He was on editorial board of the *Science China Information Sciences*. He was a member of several international conference program and senior program committees, such as ECML-PKDD, MobiMedia, ISCIT, ICCS, TrustCom, IJCNN, and ICSINC. His research interests include semantic communication, representation learning, reinforcement learning, machine learning, network function virtualization, and software defined networking.



UAV communication, and mmWave communication.

Yanping Liu received the BE degree in electronic and information engineering and the ME degree in communication engineering from Chongqing University, Chongqing, China, in 2006 and 2009, and the PhD degree in information and communication engineering from Southwest Jiaotong University, Chengdu, China, in 2018. He is currently an associate professor with the School of Big Data Statistics, Guizhou University of Finance and Economics. His research interests include game theory, learning theory, and optimization theory for the resource management on



Xi Yu received the MS degree from the School of Information, Guizhou University of Finance and Economics. She is currently an associate professor with the School of Information, Guizhou University of Finance and Economics. She is an expert in the Guian New Area. She is also a member of the AIS Asia Pacific Information Systems Special Group and a director of the Guizhou Society for Scientific and Technical Information. Her research focuses on data mining and feature extraction.



Kaiju Li received the PhD degree from the School of Computer Science and Technology, Chongqing University, Chongqing, China, in 2023. She is currently a lecturer with the School of Information, Guizhou University of Finance and Economics. Her current research interests include federated learning, edge computing, and privacy-preserving.



Lexi Xu received the PhD degree from the Queen Mary University of London, London, U.K., in 2013. He is now a senior engineer with Research Institute, China United Network Communications Corporation (China Unicom). He is also a China Unicom delegate in ITU, ETSI, 3GPP, CCSA. His research interests include Big Data, self-organizing networks, satellite system, radio resource management in wireless system, etc.